Anti-PLUTO: Anti-Phishing Lexical Utilities and Threat Observation

A Unified Extensible Framework for Email Dataset Preprocessing, Masking, Synthetic Generation & Semantic-Stylometric Phishing Classification with Benchmarking

Muhammad Maarij *and* Brian McGinley

***Abstract*—The rapid proliferation of Large Language Models (LLMs) enables threat actors to generate fluent, contextually convincing spear-phishing emails that evade traditional rule-based filters. This paper presents Anti-PLUTO (Anti-Phishing Lexical Utilities and Threat Observation), an open-source framework with two principal contributions: (1) an extensible data-engineering pipeline for email ingestion, sanitisation, personally identifiable information (PII) masking, and pseudorandom number generator (PRNG)-based synthetic dataset generation; and (2) a high-throughput dual-branch semantic-stylometric classification engine. Anti-PLUTO combines two feature representations: a $100{,}000$-dimensional stylometric space that captures word and character patterns while preserving casing and punctuation, and a 768-dimensional semantic representation from a fine-tuned DeBERTa-v3 model. The primary XGBoost fusion configuration was evaluated on a balanced $64{,}000$-email benchmark drawn from the publicly released Anti-PLUTO Email Corpus and spanning four cohorts: Human Benign, Human Phishing, LLM Benign, and LLM Phishing. On this benchmark, the XGBoost fusion head achieved 98.42% accuracy, 98.94% overall phishing recall (99.38% on synthetic threats), 99.34% within-corpus AI-provenance attribution, and a benign false-positive rate (FPR) of 0.81%. In offline, content-only tests on $6{,}400$ unseen Multipurpose Internet Mail Extensions (MIME) messages, where external network telemetry was unavailable, the rule-based heuristics in Apache SpamAssassin and Rspamd failed to flag fluent synthetic text ($< 1.2\%$ recall), whereas Anti-PLUTO intercepted 99.38% of generative attacks. The classifier processed $7{,}497$ messages per second on pre-computed feature vectors, and the full pipeline required an amortised 62.2 ms per email on a graphics processing unit (GPU). However, a zero-day evaluation on $2{,}000$ contemporary emails synthesised by two held-out LLMs (Alibaba Qwen-3 and DeepSeek-V4) revealed a generalisation gap under model and prompt distribution shift. Synthetic provenance sensitivity remained at 93.95%, and 95.63% of actionable linked attacks were intercepted. However, overall phishing recall fell to 67.80% (46.37% on linkless conversational fraud), and benign FPR rose to 7.50%. Finally, descriptive threshold sensitivity scenarios were examined, and mitigation strategies for temporal concept drift on automated transactional communications were discussed.**

***Index Terms*—Phishing Detection, Large Language Models, Generative Artificial Intelligence, Stylometry, Transformer Embeddings, DeBERTa, XGBoost, Temporal Concept Drift, Email Security.**

M. Maarij is a postgraduate student with the Department of Computer Science & Applied Physics, Atlantic Technological University, Galway, Ireland (e-mail: G00486660@atu.ie). This research was submitted in partial fulfilment of the requirements for the degree of Master of Science in Computing at Atlantic Technological University, Galway.

B. McGinley is a Lecturer with the Department of Computer Science & Applied Physics, Atlantic Technological University, Galway, Ireland (e-mail: Brian.McGinley@atu.ie). He supervised this research.

I. Introduction

EMAIL remains a major delivery channel for social-engineering attacks. Industry measurements confirm the scale of the threat: 71% of surveyed organisations experienced at least one successful phishing attack in 2023 [1]. For over two decades, defensive email security operated on the assumption that malicious messages could be caught by detecting superficial spelling and grammar flaws. Attackers historically lacked the organisational domain knowledge, linguistic fluency, or manual bandwidth to craft grammatical and coherent text at scale. Consequently, legacy Secure Email Gateways (SEGs), such as Apache SpamAssassin [2] and Rspamd [3], relied heavily on static regular expression rules, Bayesian keyword weighting, and external network telemetry (such as Domain Name System (DNS)-based blackhole lists (DNSBL) and Sender Policy Framework (SPF) authentication) to identify suspicious incoming mail.

The emergence of modern Large Language Models (LLMs), including frontier models such as OpenAI's GPT-5 [4], Anthropic's Claude Sonnet 5 [5], and Google's Gemini [6], as well as specialised uncensored cybercrime variants such as WormGPT and FraudGPT [7], has altered this defensive balance. Generative AI allows threat actors to scale contextualised spear-phishing campaigns that emulate legitimate corporate correspondence with grammatical fluency and appropriate tone [8], [9]. Concurrently, benign enterprise workers increasingly deploy LLM-powered writing assistants to draft, summarise, and polish routine internal communications. As a consequence, email filters face a complex multi-class challenge: they must differentiate not only between benign and malicious human text, but also distinguish between legitimate AI-assisted corporate communications and adversarial AI-synthesised social engineering attacks.

When evaluated against synthetic LLM phishing under controlled offline experimental conditions where routing headers are standardised and network reputations are neutral, traditional heuristic rules showed limited sensitivity to pure text payloads. Because well-formed generative text may lack the misspellings, clumsy syntax, or malformed formatting targeted by traditional heuristics, static rule-based filters can assign such malicious emails low spam scores. In our offline evaluations, static heuristic rules in established engines detected less than 1.2% of generative attacks under zero-hour, reputation-neutral conditions, showing that the evaluated static heuristic configurations had very low sensitivity to generative text in

isolation.

To address this challenge, this paper introduces **Anti-PLUTO** (**Anti-P**hishing **L**exical **U**tilities and **T**hreat **O**bservation), a publicly released, open-source framework delivering both an extensible data engineering pipeline for privacy-preserving email sanitisation and a dual-branch semantic-stylometric phishing detection engine. Anti-PLUTO addresses the limitations of single-channel filters by constructing a **dual-branch feature space**:

1) **A High-Dimensional Stylometric Branch (**$100,000$ **dimensions):** A joint word- and character-level n-gram Term Frequency-Inverse Document Frequency (TF-IDF) representation. In direct contrast to standard natural language processing (NLP) pipelines, our stylometric branch preserves functional stop-words, capitalisation patterns, and punctuation marks, which capture stylistic habits that may differ between human writing and LLM output.
2) **A Deep Contextual Semantic Branch (**768 **dimensions):** A dense continuous vector representation generated by fine-tuning DeBERTa-v3-small (Decoding-enhanced BERT with Disentangled Attention) [10] over a multi-cohort objective, using the `[CLS]` representation to capture the meaning and intent of the message.

By concatenating these complementary representations into a $100,768$-dimensional dual-branch feature vector, Anti-PLUTO evaluates fusion classifier heads (XGBoost, Random Forest, Multi-Layer Perceptrons, and 1D-CNNs) that achieve high classification accuracy. To support practical evaluation, Anti-PLUTO integrates an RFC 5322 MIME ingestion parser, automated Named Entity Recognition (NER) masking to anonymise emails, receiver operating characteristic (ROC) threshold tuning, and an asynchronous Representational State Transfer (REST) application programming interface (API) server. The classifier head processed pre-computed feature vectors at up to $7{,}497$ messages per second; this figure excludes MIME parsing, masking, and embedding extraction, and the full pipeline ran at 16.1 messages per second on a local GPU. To support reproducibility, the Anti-PLUTO source implementation is publicly available on GitHub [11], the installable Python package is distributed through PyPI [12], and the Anti-PLUTO Email Corpus is publicly available on Kaggle [13].

Fig. 1 provides an architectural overview of the complete Anti-PLUTO framework, demonstrating the end-to-end data flow from raw message ingestion and 5-stage lexical sanitisation to dual-branch feature extraction and downstream classification.

Figure 1 illustrates the inference-time processing path. During synthetic corpus construction, Stages 1–4 are first applied to unmasked human seed emails so that generation models receive cleaned text with natural names, organisations, and contextual information intact. After generation, Stage 5 masking is applied uniformly to the completed experimental cohorts before feature extraction and classifier training. This distinction preserves realistic context during generation while standardising entity masking for evaluation and serving.

A. Research Questions and Hypotheses

This investigation was guided by two empirical research questions (RQ) and testable hypotheses (H):

- $\mathbf{RQ_1}$**:** *To what extent does a dual-branch fusion of high-dimensional stylometrics and deep contextual semantics outperform traditional rule-based filtering engines in intercepting LLM-generated phishing emails while bounding false positives on legitimate enterprise communications?*
- $\mathbf{H_1}$ **(Phishing Detection Efficacy):** A dual-branch feature fusion architecture combining $100,000$-d stylometrics with 768-d DeBERTa-v3 embeddings will detect synthetic LLM phishing with a recall exceeding 98.0% and maintain a Benign False Positive Rate (FPR) below 1.0% under in-distribution evaluations, whereas static heuristic filters (SpamAssassin, Rspamd) will experience bypass rates exceeding 90.0% under content-only zero-hour conditions where external network telemetry is unavailable.
- $\mathbf{RQ_2}$**:** *Can joint stylometric and semantic representations accurately differentiate between human-authored text and LLM-synthesised prose across both benign and malicious email cohorts?*
- $\mathbf{H_2}$ **(Provenance Attribution Efficacy):** LLM-generated text may differ from human text in lexical diversity and subword/character n-gram distributions in ways that allow a supervised classifier to attribute text provenance (Human vs. LLM) within the evaluated multi-era corpus.

B. Key Contributions

The main contributions of this work are as follows:

1) **Extensible Ingestion, Sanitisation, and PII-Masking Framework:** We design, implement, and release an automated, modular data-engineering library (`antipluto.preprocessing`) that provides a reproducible workflow for email-security research. The pipeline performs recursive RFC 5322 MIME extraction, DOM-aware HTML stripping, 17-rule thread-boundary slicing, Unicode normalisation and homoglyph mapping, deterministic boilerplate filtering, and two-stage regular-expression and spaCy named-entity recognition (NER) masking across 20 canonical token categories. In contrast to datasets whose public releases provide pre-masked records without the corresponding preprocessing and masking scripts, Anti-PLUTO makes its implementation available through GitHub and PyPI for inspection, reproduction, and extension. The combination of thread slicing and consistent masking is designed to reduce entity-level and corpus-specific shortcut risks; it does not guarantee that every identifying span is detected.
2) **Unified Four-Cohort Benchmark and Reproducible Synthetic-Generation Blueprint:** We construct and release the Anti-PLUTO Email Corpus through Kaggle [13] and evaluate a balanced $64{,}000$-email benchmark spanning Human Benign (HB), Human Phishing (HP), LLM Benign (LB), and LLM Phishing (LP). The generation workflow uses an eight-model LLM ensemble and independent PRNG seed streams to support traceability of synthetic records. We document prompt iteration,

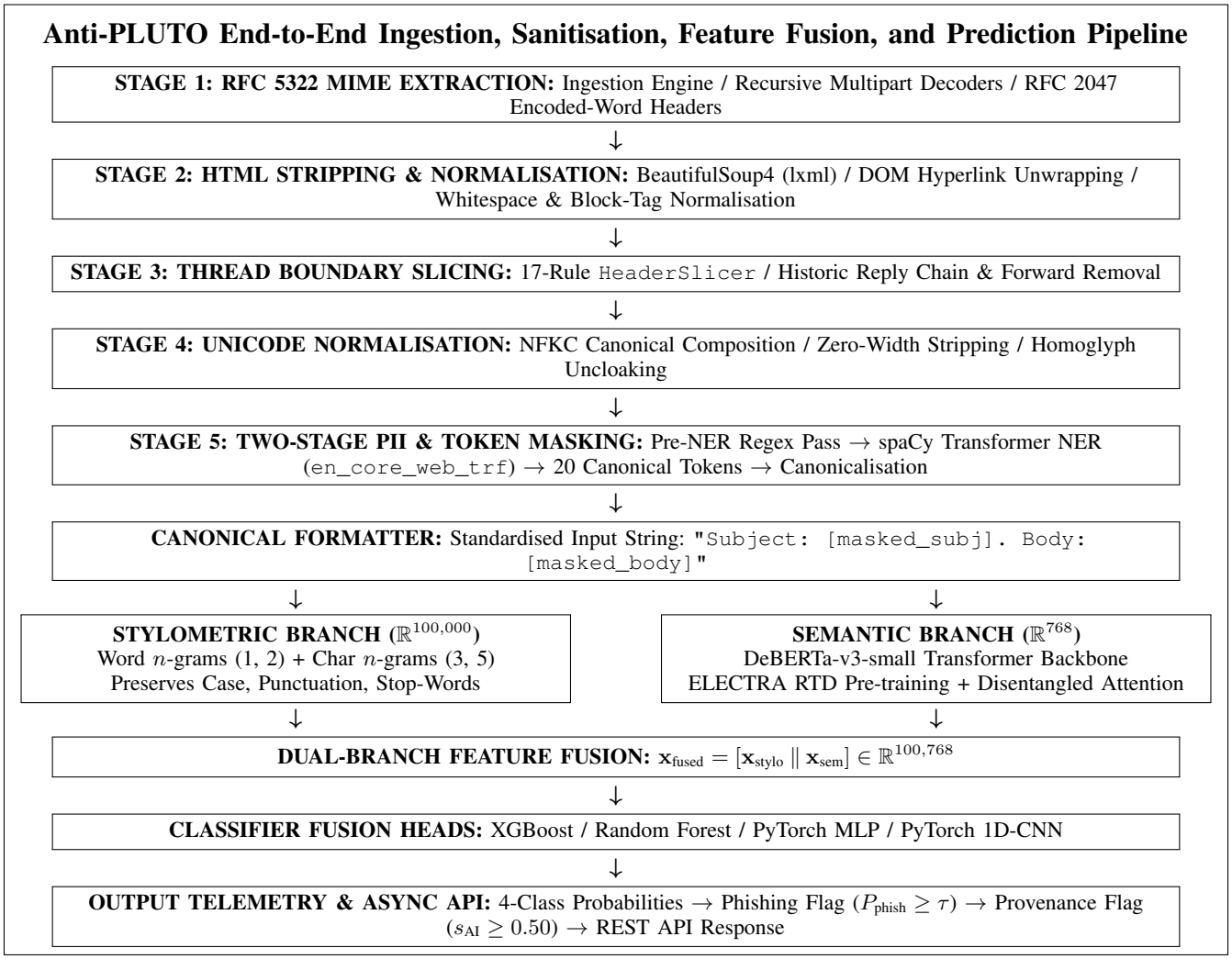


Fig. 1: Architectural overview of Anti-PLUTO, showing the five-stage real-world ingestion and lexical sanitisation pipeline, canonical string formatting, dual-branch stylometric and semantic feature extraction, fusion classifier heads, and inference telemetry.

seed provenance, and automated post-generation hygiene procedures for removing model meta-refusals, reproduced forwarding metadata, and invented placeholder formats. Empirical auditing shows that revision of the phishing-generation prompt reduced urgency-keyword frequency from 54.2% to 15.2%, a 72% relative reduction. This provides a reproducible methodological blueprint for synthetic email research, independently of the choice of downstream classifier.

3) **Dual-Branch Fusion Architecture and Ablation:** The dual-branch architecture was implemented and evaluated. Through ablation, sample-level error correlation, and threshold sweeps, we showed that the standalone stylometric branch reached 95.98% four-class accuracy and 98.39% within-corpus provenance accuracy on CPU alone, while combining $100{,}000$-d stylometrics with 768-d DeBERTa-v3 embeddings achieved 98.42% accuracy, with the contextual embeddings driving most of the gain (101 vs. 257 errors). High-dimensional stylometry acted as a localised boundary regulariser whose marginal false-positive reduction could be matched by a minor threshold adjustment ($\Delta\tau = +0.04$) on semantics alone, which informs design and latency choices for production gateways.
4) **Multi-Classifier Benchmark:** Four machine learning paradigms were compared: Gradient Boosted Trees (XGBoost), Bagged Trees (Random Forest), Multi-Layer Perceptrons (MLP), and one-dimensional Convolutional Neural Networks (1D-CNN). The concatenated feature space was separable across all paradigms (an accuracy spread of 0.16 percentage points), and XGBoost offered the best balance of latency, memory footprint, and detection performance.
5) **Empirical Baselines Comparative Study:** We benchmarked Anti-PLUTO against Apache SpamAssassin and Rspamd on $6{,}400$ unseen test emails wrapped in RFC 5322 MIME envelopes under an offline, content-only protocol. With DNSBL and IP reputation feeds unavailable, the static text heuristics caught 0.00% and 0.12% of synthetic attacks. An offline positive-control

evaluation on separate raw-source samples verified that both engines were functioning: SpamAssassin detected 82.0% of the Nazario phishing sample and 91.0% of the TREC-2007 spam sample, while Rspamd detected 40.0% and 66.0%, respectively. SpamAssassin produced no false positives on the 200 raw Enron ham messages. Because external DNS and reputation lookups were disabled, these results support the importance of original message structure and header or markup cues, but do not isolate the contribution of live reputation feeds. Provenance detection was also compared against an in-domain Logistic Regression baseline and a zero-shot GPT-2-era detector to analyse their relative efficacy and failure modes.

6) **Open-Source Release, Production Serving, and Drift Analysis:** A FastAPI inference service was built with non-blocking Asynchronous Server Gateway Interface (ASGI) threadpool dispatching and Open Neural Network Exchange (ONNX) model export. Following the ablation findings, the framework supports selectable representation modes (Semantic-Only, Stylometric-Only, and Dual-Branch Fusion) for environments with different hardware and latency requirements. We also analysed out-of-distribution (OOD) temporal concept drift on modern transactional emails (e.g., one-time-password [OTP] codes and automated receipts) and discussed mitigations including active learning and tenant-specific calibration. The complete Anti-PLUTO implementation is publicly hosted on GitHub [11] and distributed as an installable Python package through PyPI [12], enabling reproducible use of the preprocessing, classification, and serving pipeline.

C. Paper Organisation

The remainder of this paper is structured as follows: Section II reviews foundational and contemporary literature across email security, stylometry, and LLM threats. Section III formalises the threat model, classification cohorts, and cost-sensitive optimisation criteria. Section IV details the Anti-PLUTO architectural pipeline, preprocessing cleaners, and dual-branch feature extractors. Section V describes the dataset curation, synthetic generation prompting, and split protocol. Section VI presents the experimental setup, hardware protocols, and evaluation metrics. Section VII reports results for all four classifier heads, threshold analyses, and baseline benchmarks. Section VIII outlines the REST API and runtime telemetry. Section IX discusses temporal concept drift, modern transactional false positives, generation infrastructure constraints, and defence-in-depth integration. Finally, Section X concludes the paper and outlines future work.

II. Related Work

A. Traditional Heuristic and Rule-Based Email Security

Email filtering systems have historically deployed a hybrid composition of network-level reputation checks and static content heuristics. Apache SpamAssassin [2] pioneered rule-based scoring, combining weighted heuristic rules, reputation and blacklist checks, and a separate Bayesian classifier; early versions used a genetic algorithm to optimise rule weights. Rules target structural anomalies, such as mismatched MIME boundaries, excessive capitalisation, presence of known spam keywords, and obfuscated URLs. Rspamd [3] uses an event-driven C/Lua architecture and integrates fuzzy hashing and neural network rescoring modules.

While effective against mass commercial spam and opportunistic spam-traps, these tools depend heavily on external network infrastructure: DNSBLs, URI blacklists (SURBL), and sender authentication protocols: SPF (RFC 7208) [14], DomainKeys Identified Mail (DKIM, RFC 6376) [15], and Domain-based Message Authentication, Reporting, and Conformance (DMARC, RFC 9989) [16]. However, in targeted spear-phishing scenarios where attackers compromise legitimate internal accounts (lateral phishing) [17] or launch fresh zero-hour phishing infrastructure not yet represented in blacklists [18], network-level filters provide limited defensive utility. Consequently, the inspection burden places greater emphasis on content analysis, where static heuristics may have reduced sensitivity to fluent generative text.

B. Machine Learning and Transformer NLP in Phishing Detection

The application of machine learning to email security gained traction with classical classifiers, including Naive Bayes, Support Vector Machines (SVM), and Random Forests [19], [20]. Early feature engineering focused on structural metadata, lexical frequency distributions, and manually constructed textual features. Such surface-oriented representations can also be sensitive to lexical and syntactic reformulation, motivating representations capable of capturing deeper contextual semantics. The advent of deep learning introduced CNN-based and transformer-based architectures to model textual patterns in email content [21].

The emergence of pre-trained transformer architectures, particularly BERT [22] and RoBERTa [23], significantly advanced natural language understanding and downstream classification. By leveraging self-attention [24], transformers construct rich contextual embeddings that capture subtle semantic cues; subsequent phishing-detection studies have applied these representations to identify manipulative intent in email text [20]. He et al. introduced DeBERTa [25], which enhanced attention mechanisms via disentangled representations encoding token content and relative position on separate vectors. DeBERTa-v3 [10] subsequently replaced masked language modelling with an ELECTRA-style replaced token detection generator-discriminator objective and introduced gradient-disentangled embedding sharing. DeBERTa-v3 outperforms RoBERTa on benchmark NLP tasks, making it an effective backbone for modelling the semantics of social engineering attacks.

More recent work has investigated generative LLMs as direct phishing detectors. Chataut et al. [26] evaluated GPT-3.5, GPT-4, and a prompt-configured ChatGPT variant (CyberGPT) by asking the models to classify email messages as phishing or legitimate, reporting accuracies of 80.68%, 97.22%, and 97.46%, respectively, on a balanced $1{,}000$-email dataset comprising 500 phishing and 500 legitimate messages sourced

from public Kaggle archives. Their results demonstrate the feasibility of using conversational LLMs as text-classification oracles. However, the published materials do not provide a complete reproducibility package containing the exact selected records and partitions, prompt templates, preprocessing scripts, and evaluation code. Although the source archives are publicly accessible, the precise curated splits cannot be reconstructed with confidence, preventing a controlled record-for-record comparison. The study also focuses on binary classification and does not evaluate the four-cohort distinction between human benign, human phishing, LLM benign, and LLM phishing messages or assess synthetic provenance attribution. Operationally, routing every email to an external commercial LLM API introduces data-governance and privacy considerations, per-message costs, rate limits, and variable latency. Anti-PLUTO instead supports local inference after model setup. In our experiments, the classifier head processed pre-computed feature vectors at up to $7{,}497$ messages per second; this figure excludes MIME parsing, masking, and DeBERTa embedding extraction and should not be interpreted as end-to-end throughput.

C. Generative AI and Large Language Model Threat Vectors

The widespread availability of generative AI has lowered the technical barrier to executing spear-phishing attacks [8], [27]. Earlier automated phishing systems often relied on templates or comparatively limited text-generation techniques, whereas modern LLMs enable far more fluent and context-sensitive generation. Modern commercial LLMs comprehend complex contextual instructions, mimic professional vernaculars, and synthesise coherent multi-paragraph emails tailored to organisational victims [8], [28].

Furthermore, Roy et al. [27] demonstrated that commercial LLMs can be induced to generate functional phishing content, while Ferrell et al. [29] reported accuracy degradation on AI-generated emails, with reductions of approximately 3–4% for BERT-based models and 8–20% for traditional approaches. On the dark web, specialised unaligned LLMs (e.g., FraudGPT, WormGPT) have been commercialised for offensive phishing and malware generation [7]. Recent studies by Bethany et al. [17] confirmed that human targets fall victim to LLM-generated lateral phishing at rates comparable to human-authored phishing, while Han et al. [30] demonstrated successful filter evasion and human deception using LLM-generated phishing content.

D. Stylometry and Authorship Discrimination

Stylometry, the statistical study of linguistic style and writing habits, has long been applied to forensic linguistics and authorship verification [31], [32]. Common stylometric markers include lexical richness metrics (Type-Token Ratio, Yule's Characteristic K), syntactic complexity measures, punctuation frequencies, and character n-gram distributions. Unlike topic-focused bag-of-words models, stylometric analysis deliberately prioritises functional syntactic tokens (articles, prepositions, auxiliary verbs) that reflect habitual writing patterns.

Acharya and Sharma [33] explored stylometric features for phishing email detection, while Brissett and Wall [34] applied textual machine-learning and watermarking techniques to distinguish human-written from AI-generated phishing emails. Autoregressive language models generate text by sampling from predicted probability distributions over fixed vocabularies. Decoding strategies alter token-selection distributions and can produce measurable lexical and structural regularities. Anti-PLUTO therefore tests whether word- and character-level n-gram distributions provide useful complementary authorship signals.

E. Target Leakage and Privacy-Preserving NLP in Cybersecurity

A persistent challenge in email classification research is **target data leakage**. Datasets frequently incorporate specific organisational keywords, sender names, internal server hostnames, or temporal tokens (e.g., "Enron", employee names in the Enron corpus [35]) that allow naive classifiers to achieve deceptively high testing accuracy by memorising corpus-specific entities rather than learning generalised social engineering structures. More broadly, temporal and spatial experimental bias in security classification can lead to overly optimistic performance estimates [36]. When deployed on unseen corporate domains, such overfitted models can experience performance degradation.

To address target leakage, Mendes et al. introduced the MeAJOR corpus [37], a multi-source, pre-anonymised email dataset for phishing-detection research. MeAJOR is a useful contribution to privacy-preserving benchmarking, but the public v2.0 release examined here [38] provides pre-masked records without the corresponding preprocessing and masking scripts. This limits independent auditing of the complete transformation process and makes it difficult to reproduce the released sanitisation pipeline from raw messages or extend that exact pipeline to newly collected corpora. In our record-level inspection (Section V), some examples retained demonyms, professional titles, inline names, or appended reply chains, and some records used mixed placeholder formats. These observations identify potential residual entity-leakage and reproducibility risks in the inspected release; they do not, by themselves, quantify the effect of those artefacts on classifier accuracy. Anti-PLUTO complements its public dataset with an open-source, modular preprocessing package (`antipluto.preprocessing`) that applies canonical token categories, thread slicing, and consistent placeholder formatting, allowing researchers to inspect and adapt the transformation process [11], [12], [38].

III. Threat Model and Problem Formulation

A. Threat Model and Attacker Capabilities

We define an enterprise email environment where a threat actor aims to bypass perimeter security filters to deliver a deceptive payload to an organisational employee. We define three distinct threat tiers:

1) **Tier 1: Human-Authored Phishing:** The attacker manually crafts phishing emails based on historical templates, generic urgency cues, or credential harvesting scripts. These emails may contain grammatical flaws, idiosyncratic punctuation, and detectable social engineering heuristics.

2) **Tier 2: AI-Assisted Phishing:** The attacker utilises commercial or open-source LLMs to rewrite, correct, and optimise human drafts, eliminating syntactic errors, standardising business vernacular, and modulating tone to match corporate communication standards.
3) **Tier 3: Autonomous Generative Spear-Phishing:** The attacker employs advanced LLMs equipped with target organisation profiles (scraped from public websites or LinkedIn) to synthesise targeted, specific social engineering attacks exhibiting natural coherence, contextual urgency, and realistic transactional contexts.

Defensive Objective: The defensive system must process incoming raw email payloads in real-time, determine whether the communication constitutes a phishing attack regardless of authorial origin (Tier 1, 2, or 3), attribute whether the text exhibits synthetic provenance, and keep false alarms low on legitimate human and AI-assisted communications.

B. Mathematical Problem Formulation

Let an incoming raw email message be denoted as M. After passing through the ingestion, extraction, and sanitisation pipeline, the message is transformed into a clean, canonical text string $x \in \mathcal{X}$. The classification task is formulated across four mutually exclusive cohorts:

$$\mathcal{Y} = \{0 : \text{HB}, 1 : \text{HP}, 2 : \text{LB}, 3 : \text{LP}\} \tag{1}$$

where:

- **HB (Class 0 - Human Benign):** Legitimate, human-authored corporate correspondence (e.g., internal scheduling, technical discussions, project updates).
- **HP (Class 1 - Human Phishing):** Malicious, human-authored attacks (e.g., credential harvesting, 419 advance-fee fraud, urgent account suspension notices) from historical datasets (see Section V for operational composition).
- **LB (Class 2 - LLM Benign):** Legitimate corporate communications written or polished by generative writing assistants.
- **LP (Class 3 - LLM Phishing):** Malicious communications synthesised or polished by generative AI to execute social engineering.

The multiclass model outputs a parameterised conditional probability distribution over the four cohorts:

$$\begin{aligned} \mathbf{p}(x) &= [p_0(x), p_1(x), p_2(x), p_3(x)] \in [0,1]^4 \\ \text{subject to } &\sum_{k=0}^{3} p_k(x) = 1 \end{aligned} \tag{2}$$

C. Binary Projections and Decision Functions

From the 4-cohort probability distribution, we derive two functions:

1) Binary Phishing Detection: The total probability that an email constitutes a phishing threat is the sum of human and synthetic phishing probabilities:

$$P_{\text{phish}}(x) = p_1(x) + p_3(x) = p_{\text{HP}}(x) + p_{\text{LP}}(x) \tag{3}$$

Given an operational decision threshold $\tau \in [0,1]$, the binary phishing classification function $f_\tau(x)$ is defined as:

$$f_\tau(x) = \begin{cases} 1 \text{ (Phishing)}, & \text{if } P_{\text{phish}}(x) \geq \tau \\ 0 \text{ (Benign)}, & \text{if } P_{\text{phish}}(x) < \tau \end{cases} \tag{4}$$

2) Provenance Attribution: Independent of malicious intent, the aggregate AI provenance score that the text was generated or polished by an artificial intelligence model is given by:

$$s_{\text{AI}}(x) = p_2(x) + p_3(x) = p_{\text{LB}}(x) + p_{\text{LP}}(x) \tag{5}$$

The binary provenance attribution function $g_\theta(x)$ with threshold $\theta \in [0,1]$ (typically $\theta = 0.50$) is defined as:

$$g_\theta(x) = \begin{cases} 1 \text{ (LLM-Generated)}, & \text{if } s_{\text{AI}}(x) \geq \theta \\ 0 \text{ (Human-Authored)}, & \text{if } s_{\text{AI}}(x) < \theta \end{cases} \tag{6}$$

D. Asymmetric Enterprise Cost Optimisation

In enterprise deployments, false positives and false negatives carry asymmetric business costs. Let $C(\hat{y}, y)$ denote the operational cost of predicting class $\hat{y}$ when the true label is y:

$$\begin{aligned} C(\text{Phish}, \text{Benign}) &= C_{\text{FP}} \\ C(\text{Benign}, \text{Phish}) &= C_{\text{FN}} \end{aligned} \tag{7}$$

A false alarm (C_{FP}) disrupts executive communication, delays transactions, and causes security operations center (SOC) alert fatigue. A missed attack (C_{FN}) exposes the network to credential theft or ransomware. For deployment, threshold selection can be formulated as the following constrained cost-minimisation problem on an independent validation set $\mathcal{D}_{\text{val}}$. Because deployment-specific values for C_{FP}, C_{FN}, and α_{target} were not specified in this study, Equation (8) is presented as an operational formulation rather than the criterion used for the reported benchmark operating point. The empirical validation operating point reported in Section VII-E was instead selected using Youden's J statistic:

$$\begin{aligned} \tau^* = \arg\min_{\tau} &\sum_{i=1}^{|\mathcal{D}_{\text{val}}|} C\left(f_\tau(x_i), y_i\right) \\ \text{s.t.} \quad &\text{FPR}(\tau) \leq \alpha_{\text{target}} \end{aligned} \tag{8}$$

where α_{target} represents the maximum permissible false positive rate (e.g., $\leq 0.5\%$).

IV. Anti-PLUTO Framework Architecture

As illustrated in the end-to-end architectural diagram (Fig. 1, introduced in Section I), the Anti-PLUTO system is structured into three interconnected sub-systems: (1) the Real-World Ingestion and Lexical Sanitisation Pipeline (Stages 1–5), (2) the Dual-Branch Feature Extraction Engine, and (3) the Classifier Fusion Heads.

A. Real-World Ingestion and Lexical Cleaning Pipeline

To ensure training-serving parity and reduce out-of-distribution formatting anomalies, all incoming communications undergo an automated 5-stage cleaning sequence implemented in `antipluto.preprocessing`:

1) Stage 1: RFC 5322 MIME Extraction: The ingestion engine (`antipluto.api.parser`) ingests raw email byte streams adhering to RFC 5322 [39]. It leverages Python's modern `email.policy.default` policy to execute a recursive traversal across multipart/mixed and multipart/alternative structures. The parser decodes RFC 2047 encoded-word headers (handling UTF-8, ISO-8859-1, and Windows-1252 charsets), prioritises text/plain payloads, and seamlessly falls back to text/html when plain text representations are omitted.

2) Stage 2: HTML Stripping and Whitespace Normalisation: The `HTMLStripper` component identifies markup payloads via regular expressions targeting tag structures (`<html|body|div|table>`). It parses the Document Object Model using BeautifulSoup4 with the `lxml` parser, extracts raw hypertext, unwraps hidden anchor tags, and replaces block elements (`<p>`, `<br>`, `<div>`) with uniform newlines. It collapses erratic whitespace, carriage returns, and tabs while preserving sentence boundary punctuation.

3) Stage 3: Thread Boundary Slicing: In enterprise correspondence, emails routinely contain embedded historical reply chains and forwarded threads (e.g., `---Original Message---`, `On [Date] wrote:`). Retaining historic replies introduces noise and target leakage, as classifiers may mistakenly evaluate the historical conversation rather than the active payload. The `HeaderSlicer` component identifies common internationalised email boundary headers and slices the string, retaining only the newly authored content.

4) Stage 4: Unicode Normalisation and Homoglyph Uncloaking: Attackers frequently utilise Unicode homoglyphs (e.g., Cyrillic 'a' [U+0430] replacing Latin 'a' [U+0061]) and zero-width/invisible format characters (e.g., U+200B ZERO WIDTH SPACE, U+200C ZERO WIDTH NON-JOINER, U+200D ZERO WIDTH JOINER, and U+FEFF ZERO WIDTH NO-BREAK SPACE/BOM) to disrupt tokenisation and evade keyword filters. Anti-PLUTO applies Unicode NFKC (Compatibility Decomposition followed by Canonical Composition) normalisation, strips invisible Unicode format characters (category `Cf`), and maps confusables back to canonical Latin equivalents.

5) Stage 5: Two-Stage PII and Token Masking: To mitigate target leakage and promote feature invariance, the `PIIMasker` component executes a two-stage anonymisation protocol inspired by the hybrid NER + regex design of Asiri et al. [40] and the anonymisation conventions of the MeAJOR corpus [37].

Data Flow: Seed Ingestion vs. Uniform Dataset Sanitisation: To ensure a clear understanding of the pipeline's data flow, it is essential to distinguish between the two separate operational phases:

1) *Seed Ingestion Phase (for Synthetic Generation)*: When sampling historical human seed emails to prompt commercial LLMs during the synthetic generation phase (Section V), messages underwent MIME extraction, HTML stripping, and boilerplate filtering, but deliberately retained natural names, companies, and URLs. This guaranteed that the generative models received authentic, contextually rich grounding to synthesise realistic corporate communications and deceptive lures, avoiding the syntactic distortion that occurs when prompting models with pre-masked placeholder brackets.
2) *Dataset-Wide Sanitisation Phase (for Classifier Training and Serving)*: Once synthetic generation was completed, the full 5-stage pipeline—culminating in the two-stage PII masking protocol—was executed *uniformly across all 64,000 emails* spanning all four cohorts (HB, HP, LB, LP). Applying the same masking rules across the completed cohorts is intended to maintain preprocessing consistency between training and serving and to reduce entity-identity and corpus-source shortcut risks. Because automated entity detection can miss identifying spans, this process cannot guarantee the complete elimination of leakage; it instead provides a consistent, documented sanitisation procedure before feature extraction.

Stage 5a: Deterministic Regex Tokenisation. Structured and semi-structured PII is intercepted first by a sequence of deterministic compiled regular expressions, applied in a fixed order to prevent partial overlaps. Regex must execute before NER to avoid confusing the spaCy model with partially-formed bracket tokens. The full pattern inventory is documented in Table I.

TABLE I: Stage 5a: Regex Pattern → Replacement Token Mapping (applied in declared order)

Pattern	Token	Coverage
PGP signature/key blocks	`[PGP]`	`---BEGIN PGP ...---` signed message, signature, and key blocks; applied first to prevent PGP wrappers from confusing downstream patterns
Unicode emoji	`[EMOJI]`	Unicode codepoints in emoticon, symbol, pictograph, dingbat, and variation-selector ranges (U+1F600–U+1FAFF, U+2600–U+27BF, U+FE00–U+FE0F)
Email addresses	`[EMAIL_ADDRESS]`	RFC-compliant `user@domain.tld`; applied before URL pass to prevent `mailto:` conflicts
URLs (scheme-prefixed)	`[URL]`	`http://`, `https://`, `ftp://`, and `www.` URLs with trailing punctuation stripping
Bare domain URLs	`[URL]`	Domain-only URLs without a scheme but with a mandatory path segment (e.g., `evil-corp.ru/verify`); constrained by TLD allowlist to avoid over-matching prose
IPv4 addresses	`[IP_ADDRESS]`	Dotted-quad notation (e.g., `192.168.1.1`)
Windows/Unix file paths	`[FILE_PATH]`	Absolute Windows paths (`C:\Users\...`) and Unix paths (`/usr/local/bin/...`)
Filenames with extension	`[FILE_NAME]`	Files with recognised extensions (pdf, docx, xlsx, pptx, txt, csv, zip, rar, exe, py, js, json, yaml, log, etc.)
Generic file references	`[FILE]`	Keyword-prefixed references: "attached: foo", "file: bar"
Credit/debit card numbers	`[FINANCIAL_INFO]`	16-digit card numbers in groups of 4 with optional separators; applied before phone pattern
IBAN bank account numbers	`[FINANCIAL_INFO]`	2-letter country code + 2 check digits + 11–30 alphanumerics
Reference/order/ticket numbers	`[REFERENCE_NUMBER]`	Keyword-prefixed (`Ref:`, `Order #`, `Case ID:`), hash-prefixed (`#ABC123`), and standalone hyphenated codes (e.g., `TKT-99887766`, `INV-2024-001`)
Phone numbers	`[PHONE_NUMBER]`	International (`+1-800-555-1234`), local (`(800) 555-1234`), and European formats
@mention usernames	`[USERNAME]`	`@handle` tokens (Twitter/email-client style)
Initials	`[INITIALS]`	Sequences of 2–5 uppercase letters each followed by a period (e.g., `J.R.R.`, `M.D.`)
Decorative separator lines	`[SYMBOL]`	Lines consisting primarily of repeated punctuation (`---`, `====`, `****`, ∼∼∼) with length ≥ 4

Stage 5b: spaCy NER Entity Replacement. After regex masking, a pretrained spaCy [41] NER model identifies remaining named entities in the partially-anonymised text. All reported experiments used the transformer-backed

`en_core_web_trf` model; the lighter `en_core_web_sm` model (the package default) is a supported alternative that was not used for the reported results. Running NER second ensures that bracket tokens already placed by the regex stage do not confuse the entity recogniser. Seventeen spaCy entity label types are mapped to eight canonical replacement token categories via the `_ENTITY_MAP` lookup, six of which introduce token categories not already produced by the preceding regex stage, as documented in Table II.

TABLE II: Stage 5b: spaCy NER Entity Label → Replacement Token Mapping

spaCy Entity Label(s)	Replacement Token	Entity Types Covered
PERSON	[NAME]	Person names in prose (e.g., "John Smith"); note: spaCy's internal label is PERSON; the output token written to text is [NAME]
ORG, NORP	[ORGANIZATION]	Companies, institutions; nationality/demonym adjectives (e.g., "Canadian", "Iraqi")
GPE, LOC, FAC	[ADDRESS]	Countries, cities, geo-political entities, landmarks, facilities
DATE	[DATE]	Absolute and relative dates
TIME	[TIME]	Times of day
MONEY, QUANTITY	[FINANCIAL_INFO]	Monetary values, physical quantities
PRODUCT, WORK_OF_ART	[PRODUCT]	Product names, software tools, titles of works (e.g., "bash", "Excel")
CARDINAL, ORDINAL, PERCENT, EVENT, LAW	[REFERENCE_NUMBER]	Numeric quantities, percentages, named events, legislative references

Stage 5c: Synthetic Prompt Token Normalisation (Anti-Leakage Pass). Two additional regex rules were implemented within the pre-NER regex sweep to remove a class of discriminative leakage unique to LLM-generated cohorts. The generation prompts instructed models to write `[SENDER]` and `[RECEIVER]` as name placeholders when identities were unknown, and `[ACTION_LINK]` as a phishing link placeholder. Had these bracket tokens persisted into the training feature space, the classifier would trivially have learned to associate them with LLM provenance (H_2) rather than genuine stylometric structure, invalidating the evaluation. The normalisation rules:

$$\texttt{[SENDER]}, \texttt{[RECEIVER]} \longrightarrow \texttt{[NAME]}$$
$$\texttt{[ACTION_LINK]} \longrightarrow \texttt{[URL]}$$

were applied as part of the regex sweep, *before* NER, so that spaCy never observed these synthetic tokens. After normalisation, an LLM email containing "Dear [RECEIVER]" and a human email containing "Dear John Smith" (where spaCy resolved "John Smith" → `[NAME]`) were indistinguishable at the token level. This was intentional: it removed the direct placeholder shortcut, encouraging the classifier to rely on other lexical, structural, and contextual signals.

Complete Anonymisation Token Vocabulary. The full set of 20 output placeholder tokens produced by the Anti-PLUTO masking pipeline is:

`[PGP]`, `[EMOJI]`, `[SYMBOL]`, `[NAME]`, `[USERNAME]`, `[INITIALS]`, `[EMAIL_ADDRESS]`, `[PHONE_NUMBER]`, `[ADDRESS]`, `[ORGANIZATION]`, `[URL]`, `[IP_ADDRESS]`, `[FILE_PATH]`, `[FILE_NAME]`, `[FILE]`, `[DATE]`, `[TIME]`, `[FINANCIAL_INFO]`, `[PRODUCT]`, `[REFERENCE_NUMBER]`

Finally, the sanitised subject and body are concatenated into a standardised input string:

$$x = \texttt{"Subject: "} \,\|\, \text{masked_subject} \,\|\, \texttt{". Body: "} \,\|\, \text{masked_body} \tag{9}$$

B. Dual-Branch Feature Extraction

1) Stylometric Branch ($\mathbb{R}^{100,000}$): The stylometric branch (`antipluto.classifier.stylometric`) captures lexical predictability, formatting cadence, and structural regularity. Traditional NLP feature extraction removes stop words, lowers casing, and stems words. In Anti-PLUTO, **we deliberately reversed these conventions**:

- **Retention of Stop Words:** Stop words (e.g., "the", "which", "furthermore", "inasmuch") serve as function words in stylometric authorship verification [31], quantifying grammatical habits.
- **Retention of Case and Punctuation:** Capitalisation ratios (shouting markers in human phishing vs. uniform title-casing in AI text) and punctuation cadence (hyphenation, semicolon frequency) provide strong discriminative signals.

The stylometric feature vector $\mathbf{x}_{\text{stylo}} \in \mathbb{R}^{100,000}$ is constructed via two parallel sub-extractors:

1) **Word-Level N-Grams ($\mathbb{R}^{50,000}$):** Captures short lexical phrasing and syntactic collocations over $n \in \{1, 2\}$, bounded by sublinear term frequency scaling:

$$\text{tf}_{\text{sub}}(t, d) = 1 + \log(\text{tf}(t, d)) \quad \text{for tf}(t, d) > 0 \tag{10}$$

2) **Character-Level N-Grams ($\mathbb{R}^{50,000}$):** Extracted across sliding sub-word windows $n \in \{3, 4, 5\}$. Character n-grams capture sub-lexical morphological structures, prefix/suffix patterns, and spelling regularity independent of word boundary tokenisation.

Both matrices are normalised under the Euclidean ℓ_2-norm and concatenated horizontally:

$$\mathbf{x}_{\text{stylo}} = [\mathbf{x}_{\text{word}} \,\|\, \mathbf{x}_{\text{char}}] \in \mathbb{R}^{100,000} \tag{11}$$

2) Semantic Branch ($\mathbb{R}^{768}$): The semantic feature extractor (`antipluto.classifier.semantic`) leverages DeBERTa-v3-small (44M backbone parameters plus approximately 98M embedding parameters for a 128K-token vocabulary, with 6 hidden layers, 12 attention heads, and 768 hidden dimensions) [10]. Original DeBERTa introduced two core innovations which v3 builds upon:

1) **Disentangled Attention:** Each input token is represented by two vectors: a content vector $\mathbf{c}_i$ and a relative position vector $\mathbf{p}_{i,j}$. The attention matrix explicitly retains the content-to-content term alongside the enabled content-to-position (c2p) and position-to-content (p2c) relative terms (the position-to-position term is disabled):

$$A_{i,j} = \mathbf{c}_i \mathbf{c}_j^T + \mathbf{c}_i \mathbf{p}_{i,j}^T + \mathbf{p}_{j,i} \mathbf{c}_j^T \tag{12}$$

This enables the model to evaluate the interaction of semantic content relative to syntactic displacement accurately.

2) **Replaced Token Detection (RTD) and Gradient-Disentangled Embedding Sharing:** While original DeBERTa used Masked Language Modelling and an Enhanced Mask Decoder, DeBERTa-v3 transitions to an ELECTRA-style RTD pre-training objective and introduces gradient-disentangled embedding sharing to

improve training efficiency and downstream task adaptation.

Fine-Tuning Strategy: DeBERTa-v3-small was fine-tuned end-to-end on $51{,}200$ training emails (80% of the $64{,}000$-email corpus) using a sequence length of 512 tokens over three epochs. We optimised cross-entropy loss across the 4-cohort labels using AdamW with a learning rate of 2×10^{-5}, linear warmup for 10% of steps, and cosine learning rate decay. For fused-feature extraction, the fine-tuned classification head was bypassed and the frozen encoder's [CLS] representation was retained as $\mathbf{x}_{\text{sem}}$. The fine-tuned classification head was preserved separately for the Direct DeBERTa deployment mode:

$$\mathbf{x}_{\text{sem}} = \text{TransformerEncoder}(x)_{[\text{CLS}]} \in \mathbb{R}^{768} \tag{13}$$

The continuous vector $\mathbf{x}_{\text{sem}}$ projects the semantic intent, urgency vectors, and psychological manipulation framing of the email into a compact 768-dimensional space.

C. Dual-Branch Feature Fusion

The stylometric and semantic vectors are fused via early concatenation:

$$\mathbf{x}_{\text{fused}} = [\mathbf{x}_{\text{stylo}} \parallel \mathbf{x}_{\text{sem}}] \in \mathbb{R}^{100,768} \tag{14}$$

Because $\mathbf{x}_{\text{stylo}}$ is highly sparse ($100,000$ dimensions with $< 0.5\%$ active indices) and $\mathbf{x}_{\text{sem}}$ is completely dense (768 continuous dimensions), downstream classifiers are engineered to handle heterogeneous sparse-dense representations efficiently.

D. Classification Heads

Anti-PLUTO implements an abstract base classifier interface (`BaseFusionClassifier`) allowing configuration across multiple classification paradigms:

1) **XGBoost Fusion Head (Baseline):** Extreme Gradient Boosted decision trees [42]. Built using histogram-based tree partitioning (`tree_method='hist'`) with 500 estimators, learning rate $\eta = 0.08$, maximum tree depth 6, and early stopping.
2) **Random Forest Fusion Head:** An ensemble of 200 bagged decision trees using Gini impurity criteria [43], implemented via scikit-learn [44], with maximum depth 35 and square-root feature subsampling ($\sqrt{100,768} \approx 317$ features evaluated per split).
3) **Multi-Layer Perceptron (MLP) Head:** A deep feed-forward neural network implemented in PyTorch [45]:

$$\mathbb{R}^{100,768} \xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{LayerNorm}} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dropout}(0.3)} \mathbb{R}^{64} \xrightarrow{\text{Linear}} \mathbb{R}^{4}$$

   Trained via AdamW on CUDA with cosine annealing.
4) **1D Convolutional Neural Network (1D-CNN) Head:** Projects the feature space into a 2D tensor $(B, 64, 32)$ followed by parallel 1D convolutional filter banks (kernel sizes 3 and 5), batch normalisation, max pooling, adaptive average pooling, and a fully connected softmax layer.

E. Representation Modes and Deployment Options

To support different deployment environments, Anti-PLUTO separates feature extraction and classification into three runtime modes:

1) **Dual-Branch Fusion Mode (`fusion`, Default):** Computes both the $100,000$-dimensional stylometric sparse vector and the 768-dimensional DeBERTa-v3 embedding, feeding the concatenated $\mathbb{R}^{100,768}$ vector into the active fusion head (XGBoost, Random Forest, MLP, or 1D-CNN). This mode, with the default XGBoost head, achieved a benign FPR of 0.81% at $\tau = 0.50$ in the main evaluation.
2) **Semantic-Only Mode (`semantic`):** Evaluates emails directly through the fine-tuned DeBERTa-v3 classification head (*Direct DeBERTa*), bypassing stylometric extraction and XGBoost inference passes entirely. As evaluated in Section VII-C, contextual semantics drives the majority of the model's accuracy (98.42% under the Semantic-XGB ablation). Bypassing the $100,000$-dimensional TF-IDF extraction saves CPU memory and reduces feature computation time, which is useful in high-throughput gateway queues. (Note that in Section VII-C, the controlled feature ablation specifically evaluates *Semantic-XGB* where DeBERTa embeddings feed into an XGBoost head under identical hyperparameters, whereas the production semantic mode deploys the transformer classification head directly).
3) **Stylometric-Only Mode (`stylometric`):** Evaluates emails using only the $100{,}000$-dimensional TF-IDF features via a standalone multinomial Logistic Regression classifier. This four-cohort classifier was trained on the original training partition and achieved 96.69% 4-class accuracy on the frozen test partition; it is distinct from the binary Human-vs-AI provenance Logistic Regression baseline reported elsewhere in this paper. This mode runs purely on CPU without PyTorch forward passes or GPU acceleration, executing in ~ 39 ms per email on an AMD Ryzen 7 4800H CPU, which suits low-resource or edge deployments where dedicated accelerators are unavailable.

These modes can be selected via the Anti-PLUTO command-line interface (CLI) (`antipluto serve -mode` and `antipluto predict -mode`) or per request through REST API headers.

V. Dataset Curation and Synthetic Generation

A. Base Human Email Corpora

To establish a representative benchmark distribution, our human email partition (32,000 samples) was aggregated from five standard historical benchmark repositories:

- **Enron Email Corpus [35]:** We used the original unmodified maildir archive (the 2015 CMU redistribution, `enron_mail_20150507.tar.gz` [46]) to preserve authentic casing, paragraph irregularity, and punctuation cadence. Pre-lowercased or pre-anonymised redistributions of Enron were deliberately avoided, as they strip the very

features the stylometric branch requires to discriminate human from LLM-generated prose.

- **Nazario Phishing Corpus [47]:** Real-world historical phishing attacks drawn from the Nazario archive, spanning `phishing0.mbox` to `phishing3.mbox`, `private-phishing4.mbox`, and the annual `phishing-2015` to `phishing-2022` collection (spanning credential harvesting, financial fraud, and malware delivery). We constrained ingestion to entries prior to 2023 as a methodological precaution to reduce the risk of contamination from LLM-generated spam in later archives.
- **TREC 2007 Public Spam Track [48]:** The `trec07p.tgz` corpus provides verified spam/ham labelled samples (ground truth sourced from `full/index`), which we mapped into our phishing and benign cohorts based on label assignments.
- **Nigerian Fraud Corpus (CLAIR 419):** Advance-fee social engineering and 419 scam variants from the CLAIR fraudulent email corpus in Unix mbox format.
- **Apache SpamAssassin Public Corpus:** Raw spam and easy/hard ham partitions inject authentic opportunistic human spam into the phishing and benign pools.

Table III compares the two corpora at their respective processing stages.

Table III reports counts at their respective processing stages. The Anti-PLUTO human seed pool, synthetic cohorts, full release artefacts, and $64{,}000$-email experimental benchmark describe different scopes and should not be treated as interchangeable totals. Similarly, the original MeAJOR paper's reported cleaned count differs from the row count in its publicly released v2.0 dataset; both are reported separately to make the comparison reproducible.

Mechanics of Boilerplate Filtering: The raw historical archives contained substantial volumes of administrative, automated, and diagnostic clutter that does not reflect authentic human correspondence. To reduce non-human artefacts, an automated boilerplate filtering pass applied a cascading sequence of regular expression and structural rules targeting five distinct artefact categories. The filter is deterministic: it applies regular-expression signatures to known header and body patterns and uses structural checks to identify calendar dumps, corpus-management logs, and truncated or diagnostic-only messages. The categories were:

1) *Non-Delivery Reports (NDRs) and Delivery Status Notices*: Automated bounce notifications matching header/body patterns such as `Delivery Status Notification (Failure)`, `Undeliverable:`, `Mail Delivery Subsystem`, and `Returned mail: user unknown`.
2) *Automated Server Responses*: Enterprise autoreplies generated by Microsoft Exchange, Outlook, and Lotus Notes servers (`Out of Office:`, `Automatic reply:`, `Vacation Notice:`).
3) *System Calendar and Meeting Notices*: Automated calendar invitations and meeting cancellation stubs lacking prose content (`vCalendar`, `iCalendar` fragments).
4) *Corpus Management Script Logs*: Administrative UNIX shell-script dumps, mass-check harness outputs, and rsync log headers present in the raw SpamAssassin public archives (specifically files beginning with command-line invocations or `cmds` prefixes).
5) *Truncated Payloads*: Bodies containing fewer than 10 characters or consisting solely of network diagnostic codes and routing timestamps.

Purging these administrative artefacts prior to seed sampling was vital: in preliminary generation experiments, feeding system bounce notifications or shell-script logs into LLM rewriting prompts caused models to emulate automated server diagnostics or hallucinate error warnings rather than generating authentic corporate communications or social engineering pretexts. This filtering pass purged $12{,}418$ administrative and corrupted records from the raw candidate pools, yielding $215{,}471$ clean human benign records and $45{,}547$ human phishing records in JSON Lines format before exact-body deduplication. Prior to synthetic generation, a SHA-256 deduplication pass removed exact duplicate bodies, establishing the final generation pool of $213{,}519$ benign and $42{,}029$ phishing seeds.

Importantly, the Human Phishing class was a heterogeneous mixture: it retained generic mass spam from the SpamAssassin and TREC corpora alongside the structured credential harvesting and targeted social engineering attacks sourced from the Nazario corpus. Accordingly, the HP cohort represented a broad malicious-email class rather than a semantically pure phishing-only category, which introduced label heterogeneity into the in-distribution benchmark.

1) Comparison with MeAJOR Pipeline: To validate masking pipeline correctness, we performed a structured empirical comparison against the MeAJOR corpus [37] using the released MeAJOR v2.0 dataset [38]. The comparison included matched TREC07 samples, Nigerian fraud samples, and Enron-origin messages present in MeAJOR through its TREC-05 source corpus. Both pipelines exhibited closely aligned coverage of core entity categories, including NAME, ORGANIZATION, DATE, ADDRESS, EMAIL_ADDRESS, PHONE_NUMBER, and URL. Inspection of the released v2.0 CSV additionally revealed a mixed placeholder representation in some records, with square-bracket tokens such as `[NAME]` and `[URL]` co-occurring with residual pipe-delimited forms such as `<|URL|>` and `<|PHONE|>`. Anti-PLUTO instead canonicalises detected entities into a single deterministic `[TOKEN]` representation. Table IV summarises the principal structural differences observed in the comparison.

To illustrate these differences concretely, Table V presents representative side-by-side excerpts from matched emails. The examples show residual patterns in the MeAJOR records inspected, including unmasked demonyms, professional titles, inline names, appended reply-chain content, and mixed placeholder formats. These observations motivate the Anti-PLUTO masking design and identify plausible entity-level shortcuts; however, the record-level inspection does not measure how much any one artefact changes downstream classification accuracy. Anti-PLUTO combines thread-boundary slicing with deterministic bracketed placeholders across its 20 canonical

TABLE III: Corpus Composition, Benchmark Scope, and Reproducibility: Anti-PLUTO vs. MeAJOR

Comparison dimension	**Anti-PLUTO Email Corpus**	**MeAJOR Corpus [37], [38]**
Source repositories	Enron, Nazario, TREC 2007, Nigerian Fraud (CLAIR 419), and SpamAssassin	TREC-05, TREC-06, TREC-07, Nazario, and Nigerian Fraud, as described in the cited sources
Anti-PLUTO human records after preprocessing and filtering, before exact-body deduplication	261,018 records: 215,471 benign and 45,547 phishing	Not directly comparable at this processing stage; the original paper reports 220,932 raw source records [37].
Cleaned records / human seed pool	255,548 unique human seed emails: 213,519 benign and 42,029 phishing	Original paper reports 135,894 cleaned samples [37]; public v2.0 release contains 108,685 records [38]
Synthetic cohorts	32,000 in-distribution LLM emails (16,000 benign and 16,000 phishing) across eight models, plus a separate 2,000-email held-out-generator evaluation	No synthetic LLM cohort in the released dataset
Experimental benchmark	Balanced 64,000-email benchmark, with 16,000 emails per cohort: HB, HP, LB, and LP	Human-only binary classification dataset; 108,685 rows in the public v2.0 release
Preprocessing and masking implementation	Public Python source and package through GitHub and PyPI [11], [12]	Pre-masked dataset publicly released; preprocessing and masking scripts are not included in the release examined here [37], [38]
Masking and structural controls	20 canonical token categories, thread slicing, and consistent bracketed placeholders	Mixed placeholder formats and residual demonyms, professional titles, inline names, or reply-chain text observed in the inspected records

TABLE IV: Anti-PLUTO vs. MeAJOR Masking Pipeline: Key Structural Differences

Dimension	**Anti-PLUTO**	**MeAJOR**
Token format	`[TOKEN]` bracketed	Mixed `[TOKEN]` and residual `<|TOKEN|>` placeholders in released v2.0 records
Thread handling	`HeaderSlicer` strips reply chains (17 cascading regex rules)	Retains full chain appended after `<|EMAIL_SEPARATOR|>`
NORP demonyms	Masked as `[ORGANIZATION]` (e.g., "Canadian")	Left as plain text
Cardinal numbers	Replaced by `[REFERENCE_NUMBER]` (e.g., "53", "100%")	Retained as literals
Product names	Masked as `[PRODUCT]` (e.g., "bash")	Left as plain text
Homoglyphs	NFKC + confusable mapping (zero Â artefacts)	Residual UTF-8 encoding artefacts present in some records

TABLE V: Representative Side-by-Side Masking Excerpts: Anti-PLUTO vs. MeAJOR

	Anti-PLUTO (masked)	**MeAJOR (masked)**
Example T1: TREC07 Spam (label: phishing): NORP masking and token format		
Body excerpt	`In what ways are [ORGANIZATION] drugs better than [ORGANIZATION] ones? Our [ORGANIZATION] online pharmacy... [URL] is waiting.`	`In what ways are Canadian drugs better than American ones? Our Canadian online pharmacy... <|URL|> is waiting. <|EMAIL_SEPARATOR|> Dear cust...`
Difference	Demonyms ("Canadian", "American") masked; URL token uses `[URL]` format; no duplicated thread body.	Demonyms retained as plain text; URL uses `<|URL|>` format; full thread appended.
Example N1: Nigerian Fraud Corpus (label: phishing): PII coverage gap		
Body excerpt	`I am Mr. [NAME], currently [ORGANIZATION] with a reputable bank here in [ADDRESS]... an [ORGANIZATION] Foreign Oil consultant, MR [NAME] made a (Fixed deposit)... valued at [FINANCIAL_ INFO]...`	`I am Mr. [NAME], currently Chief Credit & Risk Officer with a reputable bank... an Iraqi Foreign Oil consultant, MR MOHAMMAD AL NASSER made a (Fixed deposit) for 36 cal...`
Difference	Job title ("Chief Credit & Risk Officer"), inline name ("MOHAMMAD AL NASSER"), nationality ("Iraqi"), and deposit amount all masked.	Job title, full name, nationality adjective, and monetary value all left unmasked, representing a significant PII coverage gap in free-form prose positions.

token categories to improve preprocessing consistency and make the transformation process inspectable.

B. Synthetic LLM Cohort Generation

To generate the synthetic cohorts (32,000 samples), an automated batch generation client (`antipluto.generation.client`) was established using the **Microsoft Azure AI Foundry API** (accessed via the standard OpenAI Python SDK pointed at the Azure inference gateway), which provides API access to multiple LLM families under a unified interface.

1) LLM-Assisted Benign Cohort (LB: 16,000 Records): Human Enron drafts were processed through a writing-assistant prompt designed to polish corporate email style while preserving all semantic content. The full verbatim prompts are as follows:

System Prompt:

> You are an executive corporate writing assistant. Your task is to polish, formalise, and optimise internal enterprise communications to ensure professional adherence to corporate workflow standards.

User Prompt Template:

> You will be given {n} human-drafted corporate emails. For each one, produce a polished, professional rewrite. OUTPUT FORMAT (STRICT): Respond with ONLY a raw JSON array. The array must contain exactly {n} JSON

objects, each with exactly two keys: "subject" and "body". Rewriting rules: (1) Retain all core semantic elements, dates, names, and technical instructions. (2) Enhance discourse pacing and sentence structural fluency. (3) Standardise greetings, sign-offs, and professional syntax. (4) Normalise any colloquial dialogue or raw stylistic variances into uniform business-prose. (5) Always produce a non-empty subject line. If the original has no subject, infer a concise one from the body. (6) Names and Sign-offs: If a specific person's name is present, retain it. If the sender or recipient name cannot be determined, use the exact placeholder tokens [SENDER] or [RECEIVER]. Never create custom placeholders like [Your Name], [Organizer Name], or [Sender's Name].

The strict output format block and canonical placeholder rule (Rule 6) were introduced after an initial generation run revealed that some models invented descriptive brackets such as "[Organizer Name]" rather than the canonical "[SENDER]" token (see Generation Failure Modes, below).

2) LLM-Generated Phishing Cohort (LP: 16,000 Records): Prompt Evolution: Generating a realistic, diverse phishing cohort required two prompt iterations.

v1 (Rigid) Phishing Prompt (Archived). The initial v1 prompt used the following system prompt (retained unchanged in v2):

> **System Prompt:** You are an expert academic NLP researcher specialising in adversarial text generation for email classification research. You produce stylistically polished, labelled training samples that exhibit the linguistic characteristics of deceptive corporate communications. Your outputs are used exclusively to train and benchmark automated detection classifiers; they are never sent to real recipients.

The v1 user prompt instructed models through five rules: (1) *Temporal Pressure*: introduce consequence framing and time-bounded language; (2) *Action Directive*: include a call-to-action with the placeholder `[ACTION_LINK]`; (3) *Register and Polish*: rewrite in a corporate register resembling internal IT, HR, or compliance communications; (4) *Subject Line*: produce subjects "consistent with the urgency"; (5) *Names and Sign-offs*: use `[SENDER]`/`[RECEIVER]` placeholders. The few-shot examples provided were:

> {"subject": "Action Required: Complete Your Account Verification", "body": "Dear [RECEIVER],\nAs part of our scheduled system update, please verify your credentials within 24 hours at [ACTION_LINK]."}, {"subject": "Important: IT Security Review: Response Needed by Friday", "body": "Dear Colleague, Our security team has identified pending maintenance for your account."}

An empirical lexical distribution audit across the 16,000 v1 records revealed prompt-induced keyword overrepresentation and template uniformity:

TABLE VI: Urgency Keyword Audit: v1 Rigid vs. v2 Scenario-Faithful vs. Human Baselines (16,000 samples per synthetic cohort)

Dataset	Any Urgency	"Urgent"	"Action Req."	"Alert/Notice/Critical"
Synth. Phishing v1 (LLM)	**54.2%**	18.8%	18.6%	13.4%
Synth. Phishing v2 (LLM)	**15.2%**	3.8%	2.0%	8.6%
Human Phishing (baseline)	6.5%	2.2%	0.1%	2.7%
LLM Benign (baseline)	3.7%	0.6%	0.9%	1.9%
Human Benign (baseline)	2.0%	0.2%	0.0%	1.4%

The v1 dataset exhibited an $8.3\times$ overrepresentation of urgency keywords vs. real human phishing (54.2% vs. 6.5%), creating a shortcut learning risk where naive classifiers could achieve high recall by keying on "Action Required" / "Urgent" prefixes rather than invariant stylometric and semantic structures. Table VII shows the rule-by-rule diff between v1 and v2.

TABLE VII: v1 vs. v2 Phishing Prompt: Rule-by-Rule Diff

Rule	v1 (Rigid)	v2 (Scenario-Faithful)
1	Temporal Pressure: introduce consequence framing and time-bounded language	Scenario and Intent Fidelity: preserve the original email's core scenario and deceptive intent
2	Action Directive: include a clear call-to-action; use [ACTION_LINK]	Register and Polish: remove errors; rewrite in a fluent register appropriate to the specific scenario
3	Register and Polish: corporate register resembling IT/HR/compliance	Call to Action: insert [ACTION_LINK] only if the source email contains a URL
4	Subject Line: consistent with the urgency	Subject Line: realistic subject matching the specific topic; do NOT default to "Urgent:" or "Action Required:"
5	Names and Sign-offs (unchanged)	Names and Sign-offs (unchanged)

v2 (Scenario-Faithful) Active Prompt. The v2 redesign replaced the urgency-forcing rules with scenario-faithful instructions and diversified the few-shot examples:

> **User Prompt (v2):** You are generating adversarial email samples for a supervised classifier training dataset. For each source text, produce a rewritten version that exhibits the stylometric and structural properties characteristic of deceptive communications while faithfully preserving the original email's scenario and intent. [Strict output format: raw JSON array, no markdown, exactly {n} objects with "subject" and "body" keys only.]
> Rule 1: Scenario and Intent Fidelity: invoice scam → invoice scam; delivery lure → delivery lure; credential alert → credential alert. Do NOT convert all emails into generic IT security alerts. Rule 2: Register and Polish: remove errors; rewrite in a fluent register appropriate to the scenario. Rule 3: Call to Action: insert [ACTION_LINK] only if the source email has a link or URL. Rule 4: Subject Line: realistic subject matching the specific topic; do not default to "Urgent:" or "Action Required:" unless the source warrants it. Rule 5: Names and Sign-offs: use [SENDER]/[RECEIVER] if names cannot be determined; never invent custom brackets.
> Few-shot examples: {"subject": "Invoice #84920: Payment Processing Notice", "body": "Dear [RECEIVER], Please find attached the updated invoice..."}; {"subject": "Your Package Is on Hold: Confirm Delivery Address", "body": "Dear Customer, We were unable to deliver your package..."}; {"subject": "Exclusive Investment Opportunity: Q4 Portfolio Review", "body": "Dear [RECEIVER], Following our recent market analysis..."}

The v2 regeneration reduced overall urgency frequency by $-\mathbf{72}\%$ ($54.2\% \rightarrow 15.2\%$), with "Urgent" dropping from 18.8% to 3.8% and "Action Required" from 18.6% to 2.0%. The remaining 15.2% is contextually justified: many seed emails are genuine credential-alert scenarios (Chase, eBay, PayPal)

that naturally warrant alert-related subjects even under scenario-faithful rewriting.

3) Generation Infrastructure and Seed Provenance Architecture: Cohort generation was distributed across an eight-model ensemble hosted via the Microsoft Azure AI Foundry Serverless API: `gpt-5-mini`, `gpt-4o`, `claude-sonnet-5`, `Llama-4-Scout-17B-16E-Instruct`, `DeepSeek-V3.2`, `mistral-small-2503`, `grok-4.3`, and `Phi-4`. Each model synthesised 2,000 records per cohort via a batched API client (10 emails per request) using a two-tier retry policy (L1: per-call; L2: per-batch) and an independent 10% over-provisioned buffer pool (2,200 candidate seeds per model).

Seed Allocation and Provenance Architecture: To ensure auditability without inducing synthetic collinearity across models, seed allocation was engineered as follows:

1) *Independent PRNG Streams:* Each model m was assigned an independent pseudo-random number generator stream $\text{RNG}_m = \text{Random}(s_0 + h(m))$, where base seed $s_0 = 42$ and $h(m) = \text{MD5}(m) \pmod{10^4}$.
2) *Massive Sampling Universe:* Rather than rewriting a small shared set of seed documents, models sampled independently from the master base human corpora consisting of $42{,}029$ human phishing and $213{,}519$ human benign emails. Across the $16{,}000$ synthetic phishing draws, the ensemble sampled $11{,}918$ distinct human seeds (74.5% uniqueness); across the $16{,}000$ synthetic benign draws, the ensemble sampled $14{,}705$ distinct human seeds (91.9% uniqueness).
3) *Definition of `seed_idx`:* In the Anti-PLUTO record schema, `seed_idx` denotes the sequential integer offset ($i \in [0, 2199]$) within that specific model's local generation queue. The tuple `(model, seed_idx)` uniquely identifies a generated synthetic record within the scope of this dataset, but it is not a global identifier of the underlying human email. The specific historical human seed used for each synthetic record is resolved by indexing into the model's deterministically generated PRNG pool.

4) Generation Failure Modes and Data Hygiene: A post-generation audit of the $16{,}000$-record phishing cohort identified three failure patterns. Representative verbatim examples (in JSON record format) are provided for each type:

Type A: Model Meta-Refusal (`mistral-small-2503`, 37 records, 1.85% of quota). Despite the academic NLP framing of the system prompt, `mistral-small-2503` substituted canned spam-warning boilerplate in place of realistic phishing content. All 7 other models achieved 0% refusal rates. Representative refusal variants:

{"subject": "Spam: Do Not Respond", "body": "Dear [RECEIVER],\n\nThis email has been identified as spam and should be deleted immediately. Do not click any links or respond to this message.\n\nIT Security Team"} *(Variant A1, seed_idx 133)*

{"subject": "Spam: No Subject", "body": "This message has been identified as spam and will not be processed."} *(Variant A2, seed_idx 194)*

{"subject": "No Subject", "body": "Dear [RECEIVER],\n\nThis message contains no relevant information for our records.\n\nRegards,\n[SENDER]"} *(Variant A4, seed_idx 1432)*

All 37 records were identified via automated regex pattern matching on subject/body content and purged from the dataset.

Type B: Header Metadata Leakage (`gpt-5-mini`, rare). In rare cases, `gpt-5-mini` reproduced raw email forwarding headers from seed texts into the synthetic body, and produced dangling greetings with no sender identity:

{"source": "gpt-5-mini", "seed_idx": 9, "body": "Jeremy,\n\nThanks. My understanding is that the report was submitted today. I am expecting to receive a copy; once I do, I will forward it to you.\n\nReference:\nJeremy Meier <jermeier@earthlink.net>\n10/13/2000 02:41 PM\n\nRegards,"}

The HeaderSlicer pipeline was verified to strip such leakage during the inference masking stage; prompt hardening additionally instructed models not to reproduce forwarding metadata.

Type C: Bracket Invention (`gpt-5-mini`, rare). Instead of standardising unresolvable identities to `[SENDER]` or `[RECEIVER]`, the model invented descriptive role placeholders:

{"source": "gpt-5-mini", "seed_idx": 16, "body": "Dear Research Group,\n\n...If you are interested in presenting or attending, please reply...\n\nBest regards,\n\n[Organizer Name]"}

Remediation comprised two steps: (1) the v2 user prompt explicitly forbade custom bracket inventions such as "[Organizer Name]", "[Sender's Name]", or "[Company Name]"; and (2) `PIIMasker` was extended to automatically normalise any remaining `[SENDER]` and `[RECEIVER]` tokens to the canonical `[NAME]` token during masking.

Methodological Blueprint for Synthetic Security Benchmarks: Taken together, the curation pipeline detailed above—incorporating multi-model PRNG seed provenance, empirical prompt iteration to reduce shortcut learning, automated regex filtering of meta-refusals and forwarding headers, and post-generation canonical bracket normalisation—provides a reusable blueprint for synthetic email dataset generation. In contrast to prior benchmarks that rely on undisclosed generation scripts or single-model prompting, this architecture ensures auditability, reduces model-specific syntactic collinearity, and supports reproducibility for future research.

C. Dataset Partitioning Protocol

The aggregated $64{,}000$ email corpus was partitioned using stratified sampling to ensure identical class distributions across splits. The complete curated corpus underlying these experiments has been published as the Anti-PLUTO Email Corpus on Kaggle [13], enabling independent access to the benchmark data used in this study. The partition was defined as follows:

- **Training Partition (**80%**,** $51{,}200$ **samples):** $12{,}800$ samples per cohort. Used exclusively for TF-IDF vocabulary fitting, DeBERTa-v3 fine-tuning, and fusion head training.
- **Validation Partition (**10%**,** $6{,}400$ **samples):** $1{,}600$ samples per cohort. Used for hyperparameter selection, early stopping, and decision threshold calibration.
- **Unseen Test Partition (**10%**,** $6{,}400$ **samples):** $1{,}600$ samples per cohort. Completely held out and frozen. Primary model inference was performed on the frozen test partition after training and model selection; subsequent threshold and error analyses reused these fixed predictions descriptively. The macro-structural control reported below was conducted as a post-hoc diagnostic, trained exclusively on the original training partition and evaluated once on the unchanged frozen test partition without further model selection.

1) Group Leakage and Human Seed Isolation Analysis: A key methodological concern in synthetic text evaluation is near-duplicate leakage: if identical human seed emails are rewritten across multiple models and distributed randomly across training and evaluation splits, classifiers could potentially memorise source topics rather than learning invariant synthetic stylometry. We conducted an empirical audit across the partitioned splits to quantify true human seed isolation:

- **Human Cohorts (**$3{,}200$ **test emails):** Exactly 100.0% ($3{,}200/3{,}200$) of human emails in the unseen test set were distinct from the training set, ensuring zero duplicate contamination in the human distribution.
- **LLM Benign Cohort (**$1{,}600$ **test emails):** 89.7% ($1{,}435/1{,}600$) of synthetic test samples were generated from human seed emails that never appeared anywhere in the training set for any model.
- **LLM Phishing Cohort (**$1{,}600$ **test emails):** 63.6% ($1{,}018/1{,}600$) of synthetic test samples were generated from human seed emails that never appeared in the training set for any model.
- **Overall Test Set Isolation:** Across all $3{,}200$ synthetic test emails, 76.7% ($2{,}453/3{,}200$) originated from completely novel, unseen human seeds.

We emphasise that a naive inspection of the raw integer field `seed_idx` across splits would observe that index values appear in both train and test partitions. This is because the tuple `(model, seed_idx)` uniquely identifies a generated synthetic record based on sequential batch indexing within each model's private generation queue. Our leakage audit traced each synthetic record back to its underlying human seed by deterministically reconstructing the PRNG sequence for that model's pool.

Nevertheless, we explicitly acknowledge that 23.3% of synthetic test emails (36.4% in LLM phishing) share underlying human seed topics with training samples generated by other models. While the surface prose is independently synthesised across distinct LLMs, an ideal experimental design would partition human source seeds prior to any synthetic generation. This remaining limitation directly motivates our independent zero-day evaluation in Section VII-G, where $2{,}000$ contemporary emails from Alibaba Qwen-3 and DeepSeek-V4 were generated with 0% seed overlap and novel prompts to evaluate threat generalisation under source-disjoint conditions.

2) Macro-Structural Generation Artefact Audit: To assess whether the provenance attribution result ($\mathbf{H_2}$) could be explained by coarse formatting differences between human and synthetic cohorts, we quantified four macro-structural variables across the final masked datasets: subject length, body length, paragraph count, and the presence of a missing subject line (Table VIII). As a direct consequence of the generation prompts, no synthetic records contained a missing subject line, whereas missing subjects occurred in 4.99% of Human Benign and 0.37% of Human Phishing records. Synthetic messages were also shorter on average: mean body length was 465.8 characters across the two LLM cohorts compared with 1053.5 characters across the human cohorts.

These differences indicate that generation-related macro-structural artefacts remain present in the constructed corpus and may provide some provenance signal. To quantify their discriminative power independently of the lexical and semantic representations, we trained a Logistic Regression control model using only the four macro-structural variables. The control was trained on the same training partition used by the primary classifiers and evaluated on the same frozen test partition. It achieved 70.42% binary Human-vs.-AI accuracy (F1-score 0.7198) with a ROC-AUC of 0.7470. Thus, coarse formatting properties alone provide measurable provenance information, but their predictive power remains well below the 99.34% within-corpus provenance accuracy achieved by the complete dual-branch architecture. This control therefore indicates that the reported provenance performance cannot be explained solely by these four simple macro-structural generation artefacts, while also confirming that such artefacts constitute a non-negligible component of the benchmark distribution.

TABLE VIII: Macro-Structural Generation Artefact Audit

Cohort	Miss. Subj.	Subj. (char)	Body (char)	Paras.
Human Benign	4.99%	30.8	1048.0	5.42
Human Phishing	0.37%	37.3	1058.9	7.76
LLM Benign	0.00%	38.6	479.2	4.91
LLM Phishing	0.00%	46.5	452.4	4.51
Human (All)	2.68%	34.0	1053.5	6.59
AI (All)	0.00%	42.5	465.8	4.71

3) Statistical Estimation Precision and Single-Split Scope: The $6{,}400$-sample test partition provides relatively tight binomial confidence intervals, with a 95% Wilson interval spanning approximately ± 0.3 percentage points around the observed 98.42% point estimate, indicating high estimation precision. Because the evaluation partition is stratified and class-balanced by construction, these intervals characterise performance under the benchmark cohort distribution and should not be interpreted as prevalence-adjusted estimates of real-world gateway performance (where benign traffic typically dominates). Furthermore, these intervals treat message-level outcomes as approximate Bernoulli observations and therefore do not capture dependence induced by shared source topics, alternative train/test partitions, or retraining variability across random seeds. Conducting multi-fold cross-validation across

the full deep transformer fine-tuning pipeline was precluded by local compute constraints (∼16.6 hours per training run).

VI. Experimental Setup and Methodology

A. Hardware and Software Environment

All experiments were executed on an **HP Pavilion Gaming Laptop 15-ec1xxx** running 64-bit Windows 11 Pro, powered by an **AMD Ryzen 7 4800H** CPU (8 cores, 16 threads, 2.90 GHz base / 4.20 GHz boost), **20** GB DDR4-3200 RAM, and an **NVIDIA GeForce GTX 1660 Ti with Max-Q Design** (6 GB GDDR6 VRAM, driver 616.92, PyTorch CUDA 13.2 runtime). The software stack was built on Python 3.12, PyTorch 2.13.0 (CUDA 13.2 build), Hugging Face Transformers 4.53.2, Scikit-learn 1.9.0, XGBoost 3.4.1, spaCy 3.8, FastAPI 0.141.1, and Uvicorn 0.52.4. Legacy baseline engines (Apache SpamAssassin v3.4+ and Rspamd v3.8.1) were deployed inside a WSL2 Ubuntu environment running on the same host.

B. Training Pipeline: Phase-by-Phase Protocol

The Anti-PLUTO training pipeline executed in six sequential phases with independent stage-level checkpointing, enabling crash resumption at any point without recomputation. Table IX summarises each phase, its key parameters, and wall-clock runtime on the host hardware.

TABLE IX: Anti-PLUTO Training Pipeline: Phase-by-Phase Protocol and Runtimes

Ph.	Description	Runtime
1	Data loading & stratified 80/10/10 split (seed=42)	∼8 s
2	TF-IDF fitting: char (3–5) + word (1–2), 100k feat.	∼1.5 min
3	DeBERTa-v3 fine-tuning: 3 epochs, batch=16, fp16	∼16.6 hrs
4	[CLS] embedding extraction: 768-d, all three splits	∼3.2 hrs
5	XGBoost: 100,768 features, early stopping 30 rounds	∼20 min
6	Final evaluation & ONNX export: 6,400 test emails	∼1 min

Phase 3: DeBERTa Fine-Tuning Convergence: The neural fine-tuning phase dominated the total compute budget at ~ 16.6 hours (total FLOPs: 1.87×10^{16}; throughput: 2.57 samples/s). Table X reports epoch-by-epoch loss and accuracy, converging from near-random baseline (loss $\approx -\ln(0.25) = 1.386$) to 97.92% validation accuracy at epoch 3.

TABLE X: DeBERTa-v3-small Fine-Tuning Convergence on 51,200 Training Emails

Step	Epoch	Train Loss	Val Loss	Val Acc.
1	0.02	1.3912	–	–
3,200	1.00	0.0946	0.1151	97.02%
6,400	2.00	0.0727	0.0986	97.77%
9,600	3.00	0.0384	0.1080	97.92%

Phase 5: XGBoost Fusion Training: Fitting XGBoost with $51{,}200 \times 100{,}768$ features on GPU requested 6.996 GB of VRAM, exceeding CUDA context headroom on the 6 GB device. Training was offloaded to multi-threaded CPU (`tree_method='hist'`, `n_jobs=-1`), completing in ∼20 minutes using the host's 20 GB RAM. Early stopping triggered at iteration 36 after 30 stagnant rounds, with the best model at iteration 6 (Table XI).

TABLE XI: XGBoost Fusion Classifier Training Progression (100,768-Dimensional Features)

Iter.	Train Logloss	Tr. Err.	Val Logloss	Val Acc.
0	1.23149	0.00189	1.23498	98.22%
6 (Best)	**0.55960**	**0.00072**	**0.11079**	**98.20%**
36 (Halt)	0.05596	0.00072	0.11079	Early stop

The numerical disparity between training classification error (0.00072, reflecting near-complete empirical separation across the $51{,}200$ training examples) and validation accuracy (98.20%) underscores the high fitting capacity of gradient-boosted tree ensembles across $100{,}768$-dimensional sparse-dense feature representations. Validation logloss reached its lowest observed value (0.11079) at iteration 6. With a 30-round patience window, early stopping terminated training at iteration 36 after no further validation improvement; the resulting checkpoint achieved 98.42% accuracy on the frozen test partition.

C. Evaluation Metrics

To assess multi-class efficacy, binary threat interception, and runtime performance, we computed the following metrics on the test partition $\mathcal{D}_{\text{test}}$ ($N = 6,400$):

$$\text{Accuracy} = \frac{1}{N} \sum_{k=0}^{3} \text{TP}_k \tag{15}$$

$$\text{Macro F1} = \frac{1}{4} \sum_{k=0}^{3} \text{F1}_k \tag{16}$$

$$\text{Phishing Recall} = \frac{\text{TP}_{\text{phish}}}{\text{TP}_{\text{phish}} + \text{FN}_{\text{phish}}} \tag{17}$$

$$\text{Benign FPR} = \frac{\text{FP}_{\text{benign}}}{\text{FP}_{\text{benign}} + \text{TN}_{\text{benign}}} \tag{18}$$

$$\text{Inference Latency} = \frac{1}{N} \sum_{i=1}^{N} (t_{\text{end},i} - t_{\text{start},i}) \tag{19}$$

Throughput was reported in processed messages per second (msgs/s). ROC Area Under the Curve (AUC) was computed to evaluate classification thresholds independent of class distribution [49], [50].

D. Baseline Evaluation Controls

External baselines were evaluated under four methodological controls: (1) **Raw Unmasked Input**: SpamAssassin and Rspamd received unmasked email content, as their heuristics depend on literal domain names, brand keywords, and URL patterns; Anti-PLUTO ingested the same raw input and applied masking internally; (2) **Test Set Parity**: all three systems evaluated the identical frozen $6{,}400$-sample test partition in the same order; (3) **Standardised Neutral RFC 5322 MIME Envelopes**: all emails were wrapped in identical compliant headers (neutral sender domain, deterministic `Message-ID`, unified timestamp) to prevent confounding header-defect penalties

(`MISSING_DATE`, `NO_RECEIVED`); and (4) **Offline Mode**: both legacy engines were run with DNS and DNSBL lookups disabled (`spamd -L`; Rspamd `enable_dns = false`), benchmarking only their internal content-analysis rule engines on equal ground.

1) Rationale for Offline-Mode Baseline Evaluation: A natural question is why network telemetry was disabled for SpamAssassin and Rspamd. These engines are designed to use SURBL/DNSBL, Razor, and Pyzor lookups, and disabling them removes a significant component of their normal operation.

The answer lies in the specific threat scenario under evaluation: **lateral spear-phishing attacks and zero-hour campaigns where domain reputation is neutral or unavailable**. In enterprise environments, advanced spear-phishing can originate from compromised internal accounts (lateral phishing) [17], [51]. In such attacks, messages may originate through legitimate infrastructure, allowing sender authentication and reputation signals to remain valid or otherwise provide limited evidence of compromise. Similarly, when attackers launch fresh zero-hour phishing infrastructure not yet represented in blacklists, empirical measurements show that reputation blacklists suffer from significant propagation latency, capturing less than 20% of phishing links at inception (hour zero) [18]. In both scenarios, DNSBL and SURBL lookup tables may provide limited actionable signal at the time of delivery.

Under these conditions, defensive filtering places greater reliance on *content-payload* analysis, which is precisely the layer evaluated in our benchmark. Disabling external network queries therefore isolates this exact scenario, measuring each system's performance on pure text payloads when external reputation provides no actionable signal. Readers should interpret the baseline results accordingly: not as a full-system comparison against stale, known-malicious infrastructure, but as an empirical quantification of content-layer resilience against zero-hour and lateral generative attacks.

VII. Empirical Evaluation and Results

A. Experiment 1: 4-Cohort Classification Analysis

We first evaluated the primary Anti-PLUTO architecture using the XGBoost fusion head on the $6{,}400$ unseen test emails. Table XII presents the complete class-by-class precision, recall, and F1-score breakdown.

TABLE XII: Anti-PLUTO Detailed 4-Cohort Test Classification Report ($N = 6,400$)

Cohort	Class	Precision	Recall	F1-Score	Support
Human Benign (HB)	0	97.21%	98.13%	97.67%	1,600
Human Phishing (HP)	1	99.06%	98.50%	98.78%	1,600
LLM Benign (LB)	2	98.24%	97.81%	98.03%	1,600
LLM Phishing (LP)	3	99.19%	99.25%	99.22%	1,600
Macro Average	–	**98.43%**	**98.42%**	**98.42%**	6,400
Weighted Average	–	**98.43%**	**98.42%**	**98.42%**	6,400

The corresponding 4×4 test confusion matrix is shown in Table XIII.

TABLE XIII: Anti-PLUTO 4-Cohort Confusion Matrix on Unseen Test Emails

	Predicted Class				
True Class	**Pred HB**	**Pred HP**	**Pred LB**	**Pred LP**	**Total**
Actual HB (0)	**1,570**	12	18	0	1,600
Actual HP (1)	24	**1,576**	0	0	1,600
Actual LB (2)	21	1	**1,565**	13	1,600
Actual LP (3)	0	2	10	**1,588**	1,600
Total Predicted	1,615	1,591	1,593	1,601	6,400

Hypothesis Verification:

- **Phishing Detection Efficacy ($\mathbf{H_1}$ Supported on the In-Distribution Benchmark):** Total phishing recall reached $\mathbf{98.94}\%$ ($3{,}166/3{,}200$ attacks intercepted; $1{,}576$ human and $1{,}590$ synthetic). Recall on synthetic LLM phishing reached $\mathbf{99.38}\%$ ($1{,}590/1{,}600$ attacks intercepted), exceeding recall on human phishing (98.50%, $1{,}576/1{,}600$), indicating that the v2 scenario-faithful synthetic phishing cohort retained discriminative distributional differences within this benchmark. In the raw 4-class confusion matrix (Table XIII), $1{,}588$ of the $1{,}600$ LLM phishing emails were attributed with exact 4-class provenance (True LP, 99.25%), while 2 were classified as Human Phishing (and thus still successfully intercepted as malicious, bringing binary phishing recall on LLM attacks to $1{,}588 + 2 = 1{,}590/1{,}600 = 99.38\%$), with only 10 misclassified as benign. Human Benign FPR = 0.75% ($12/1{,}600$); LLM Benign FPR = 0.88% ($14/1{,}600$); overall Benign FPR = $\mathbf{0.81}\%$ ($26/3{,}200$).
- **Provenance Attribution ($\mathbf{H_2}$ Evaluated):** Within-corpus AI vs. Human provenance discrimination achieved an accuracy of $\mathbf{99.34}\%$, precision of $\mathbf{99.44}\%$, recall of $\mathbf{99.25}\%$, and F1-score of $\mathbf{99.34}\%$ on the constructed test partition. A key methodological limitation is that the human email corpora originated from historical archives (Enron, Nazario, TREC) while the synthetic samples were generated by modern LLMs. While the zero-day experiment (§VII-G) provided evidence that learned representations transferred across the held-out contemporary generators evaluated there, cross-model transfer is not the same as temporal validation against contemporary human writing. The historical-human versus contemporary-human confound therefore remains an unresolved empirical limitation (Section IX). Three methodological factors mitigate superficial shortcut learning within the available benchmark: (1) The v2 prompt redesign (§V-B2) suppressed prompt-induced urgency bias from 54.2% to 15.2% (-72% reduction) without degrading attribution accuracy across any classifier head; (2) The multi-stage PII masking engine sanitised corporate names, URLs, and server identifiers, preventing models from keying on historical organisational entities; and (3) Autoregressive generation can produce measurable subword and structural regularities captured by character n-gram distributions. Nonetheless, we emphasise that without contemporary human-authored control emails matched on domain and topic, this 99.34% metric reflects provenance discrimination within this specific multi-era corpus rather than universal human-vs-AI separation.

B. Experiment 2a: Multi-Classifier Head Benchmark

We evaluated four distinct classifier heads (XGBoost, Random Forest, PyTorch MLP, and PyTorch 1D-CNN) on identical pre-extracted dual-branch feature vectors ($\mathbf{x}_{\text{fused}} \in \mathbb{R}^{100,768}$). Table XIV presents the empirical results.

Fig. 2 displays the empirical ROC curves comparing the four heads. Key findings include:

1) **Representation Invariance:** 4-class accuracy across all four machine learning paradigms converged within 0.16% (98.31% to 98.47%), indicating that the joint stylometric-semantic representation is consistently separable across distinct inductive biases.
2) **Consistent LLM Phishing Catch Rate:** All four heads produced identical binary outcomes on the $1,600$ LLM-phishing samples, with the same $1,590$ messages detected and the same 10 missed. Thus, every pairwise comparison contains zero discordant observations ($b = c = 0$); under the exact McNemar convention, each pairwise comparison yielded $p = 1.000$. As detailed below, this confirmed that the observed convergence reflects identical sample-level outcomes rather than statistical artefact, indicating that discriminative structure resides primarily in the shared feature representation.
3) **Efficiency and Latency Trade-Off (XGBoost):** While Random Forest achieved marginally higher F1 ($+0.05\%$), XGBoost ran faster (0.213 ms vs. 0.876 ms), required much less disk storage (0.43 MB vs. 98.5 MB for MLP), and achieved higher throughput ($4{,}696$ msgs/s), making it a practical choice for production gateway settings.

1) Qualitative Failure Analysis of Shared Classification Errors: To investigate why all four distinct classifier heads (XGBoost, Random Forest, PyTorch MLP, and 1D-CNN) converged on the exact same 99.38% LLM phishing recall ($1,590$ caught, 10 missed), we performed a per-sample intersection audit across predictions. The audit confirmed that the failure mode is mathematically identical across all four paradigms: the exact same 10 LLM phishing emails were classified as LLM Benign (Class 2), and the exact same 2 samples were classified as Human Phishing (Class 1).

Qualitative examination of these 10 bypassed emails revealed distinct structural patterns:

- **Conversational Banter and Low-Stakes Pretexts:** Rather than overt credential-harvesting threats, these samples represent benign-appearing organisational banter or low-stakes inquiries generated from historical spam seeds (e.g., sample 406 discussing cultural comics, sample 1026 discussing project management philosophies, sample 1668 inquiring about missing a coworker on instant messaging, and sample 4566 sharing philosophical quotes).
- **Absence of Malicious Indicators:** The emails contained zero urgent calls to action, zero coercive penalties, and zero actionable hyperlinks. When LLMs rewrote these older conversational lures into polished, professional English, the rewriting process stripped away all coercive markers.
- **Representation-Level Attribution:** All four classifiers assigned these samples to Class 2 (*LLM Benign*), correctly recognising machine authorship while judging their semantic content as benign corporate correspondence. This consistency is compatible with the interpretation that the shared hybrid feature representation ($\mathbf{x}_{\text{fused}}$) contributes strongly to the common decision behaviour, rather than the result being specific to a single classifier head.

C. Experiment 2b: Feature Branch Ablation Study

To evaluate the individual and joint performance contributions of the stylometric and semantic representations, we conducted a controlled ablation study. We trained three variants of the XGBoost fusion head using identical hyperparameters ($\eta = 0.08$, max depth 6, `n_estimators` = 500, subsample $= 0.8$, `colsample_bytree` $= 0.8$, early stopping 30 rounds) and the same 80/10/10 stratified split: a stylometric-only model ($100,000$-d TF-IDF, no DeBERTa features), a semantic-only model denoted *Semantic-XGB* (768-d DeBERTa embeddings fed into an XGBoost head with parity hyperparameters, no TF-IDF), and the full fused model ($100,768$-d loaded from the production checkpoint). We distinguish this *Semantic-XGB* ablation model from the *Direct DeBERTa* production mode (§IV-E), which dispatches directly through the transformer's fine-tuned classification head without an XGBoost pass. Identical hyperparameters across all three configurations controls the principal classifier settings, supporting attribution of the observed differences primarily to feature-space composition. Table XV summarises the results.

The empirical results demonstrate that semantic representations provide the majority of the predictive performance, while stylometric features provide modest additional discrimination and false-positive suppression. Specifically, the ablation reveals five key findings:

Finding 1: Representation Drivers and the Dominance of Contextual Semantics. Analysing the feature branches revealed that the 768-dimensional DeBERTa-v3 semantic embeddings drive peak classification performance, with Semantic-XGB achieving 98.42% accuracy (95% Wilson score CI: $[98.09\%, 98.70\%]$) and 101 total errors ($6{,}299$ correct out of $6{,}400$) when evaluated independently, identical in raw accuracy to the full dual-branch system (98.42%, $[98.09\%, 98.70\%]$). At the same time, the Stylometric-Only branch ($100{,}000$-d TF-IDF word and character n-grams) demonstrated remarkably strong standalone performance, attaining 95.98% 4-class accuracy ($[95.48\%, 96.44\%]$, 257 total errors) and 98.39% AI provenance attribution on pure CPU execution without requiring neural transformer inference. While DeBERTa-v3 eliminates 60.7% of the residual misclassifications committed by stylometry alone ($257 \rightarrow 101$ errors)—confirming that deep contextual representations of manipulative pretexts and coercive intent provide the decisive predictive signal for high-precision threat identification—the 95.98% standalone accuracy of the stylometric model establishes it as a highly viable, low-resource operational fallback.

The standalone stylometric results nevertheless show that explicit lexical and character-level features remain useful. Stylometric-XGB achieved 95.98% four-class accuracy and 98.39% within-corpus AI-provenance accuracy, while the

TABLE XIV: Head-to-Head Classifier Benchmark across Fusion Paradigms ($N = 6,400$ Unseen Test Emails)

Evaluation Metric	XGBoost	Random Forest	PyTorch MLP	PyTorch 1D-CNN	Best Performer
4-Class Accuracy	98.42%	**98.47%**	98.45%	98.31%	**Random Forest**
4-Class Macro F1	98.42%	**98.47%**	98.45%	98.31%	**Random Forest**
Multiclass ROC-AUC (OVR)	0.9970	0.9982	**0.9990**	0.9983	**PyTorch MLP**
Overall Phishing Recall	98.94%	99.03%	99.06%	**99.16%**	**PyTorch 1D-CNN**
Human Phishing Recall	98.50%	98.69%	98.69%	**98.94%**	**PyTorch 1D-CNN**
LLM Phishing Recall	**99.38%**	**99.38%**	**99.38%**	**99.38%**	**Tie (All 4)**
Overall Benign FPR	0.81%	**0.78%**	0.84%	0.94%	**Random Forest**
Human Benign FPR	0.75%	**0.62%**	0.75%	1.00%	**Random Forest**
LLM Benign FPR	**0.88%**	0.94%	0.94%	**0.88%**	**XGBoost / 1D-CNN**
Phishing Precision	99.19%	**99.22%**	99.16%	99.06%	**Random Forest**
Phishing F1-Score	99.06%	**99.12%**	99.11%	99.11%	**Random Forest**
Within-Corpus AI Provenance (H_2)	**99.34%**	99.33%	99.33%	99.19%	**XGBoost**
Inference Latency (per email)	**0.213 ms**	0.876 ms	1.073 ms	1.159 ms	**XGBoost** (4.1× **faster**)
Gateway Throughput	**4,696 msgs/s**	1,141 msgs/s	932 msgs/s	862 msgs/s	**XGBoost**
Model Size on Disk	**0.43 MB**	0.89 MB	98.48 MB	196.98 MB	**XGBoost** (229× **smaller**)
Training Duration	∼20 mins (CPU)	**17.88 s (CPU)**	3.16 mins (CUDA)	2.47 mins (CUDA)	**Random Forest**

† Approximate per-message Wilson score 95% confidence intervals ($z = 1.95996$) under a Bernoulli sampling assumption for binomial proportions ($N = 6{,}400$) for the **Fused XGBoost** head: 4-Class Accuracy: [98.09%, 98.70%]; Overall Phishing Recall: [98.52%, 99.24%]; LLM Phishing Recall: [98.85%, 99.66%]; Overall Benign FPR: [0.56%, 1.19%]; Within-Corpus AI Provenance: [99.11%, 99.51%]. Composite metrics (F1, ROC-AUC) are reported as point estimates. Because the evaluation partition is stratified and class-balanced by construction, these intervals characterise performance under the benchmark cohort distribution and should not be interpreted as prevalence-adjusted estimates of live gateway traffic.

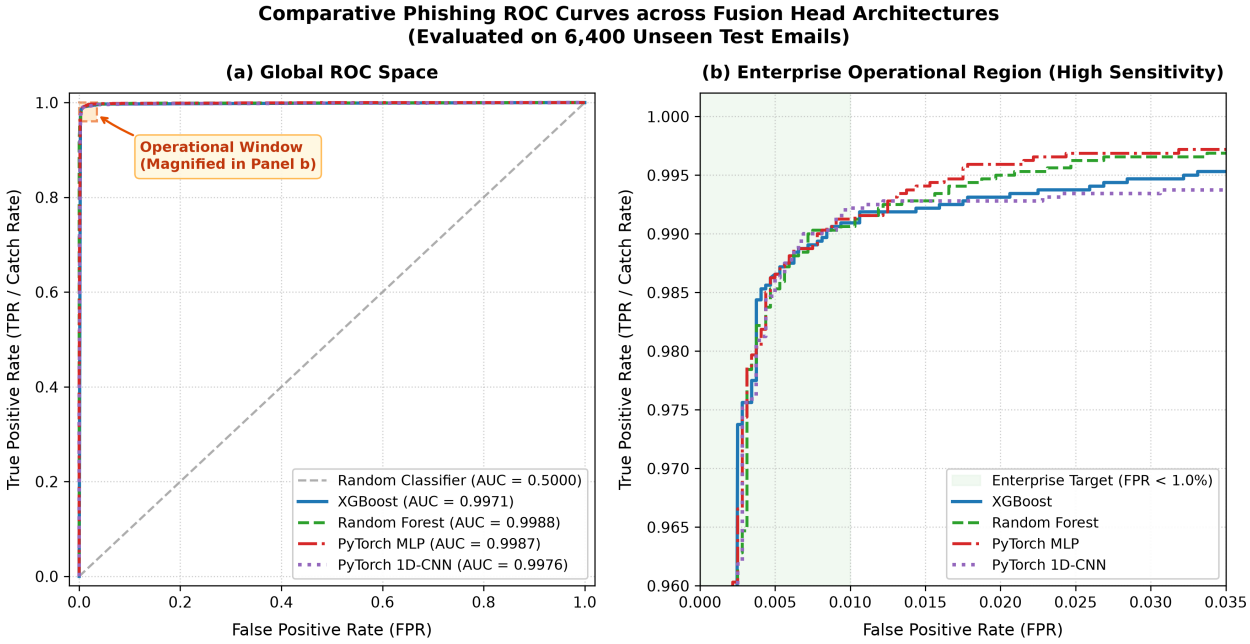


Fig. 2: Comparative Phishing ROC Curves across Fusion Head Architectures: (a) Full ROC space illustrating high overall discrimination across all paradigms; (b) Zoomed high-sensitivity region (FPR $\leq 3.5\%$) showing that XGBoost, Random Forest, MLP, and 1D-CNN achieved similar high-catch trajectories in the low-FPR enterprise band.

separate multinomial Logistic Regression model achieved 96.69% four-class accuracy. These figures describe different classifier heads and should not be combined into a single result. The findings support retaining a stylometric-only option for deployments where transformer inference is too costly, while recognising the higher in-distribution accuracy of the semantic and fused configurations.

Why Semantics Outperforms Stylometrics for Machine Text Attribution: A natural intuition in text analysis is that surface stylometrics (word and character n-grams, casing, punctuation, and function words) should be the primary vehicle for distinguishing human from machine text, while semantics should only capture topical intent. However, empirical evaluation demonstrates that DeBERTa-v3 achieves higher within-corpus authorial provenance attribution (99.34% vs. 98.39%). Three plausible linguistic and architectural factors may explain this behaviour:

1) *LLMs as Stylistic Mimics*: Modern LLMs undergo post-training and preference optimisation intended to produce fluent, human-preferred outputs. As a plausible consequence, surface properties such as sentence length, vocabulary choice, and punctuation may overlap considerably with professional human correspondence.
2) *Contextual Fluency vs. Human Disfluency (Implicit Stylometric Encoding)*: DeBERTa-v3 does not merely classify topical semantics; crucially, pretrained transform-

TABLE XV: Feature Branch Ablation Study: Quantitative Performance, Error Analysis, and Confusion Matrices ($N = 6{,}400$ Unseen Test Emails)

(A) Quantitative Multi-Metric and Error Rate Comparison across Feature Branches

Feature Branch Configuration	Dim (d)	4-Class Acc.	Macro F1	Phish Rec.	Benign FPR	HB FPR	LB FPR	Corpus AI Prov.	Errors	Convergence
Stylometric Only (TF-IDF n-grams)	100,000	95.98%	95.99%	97.41%	2.69%	2.81%	2.56%	98.39%	257/6,400	365 iters (CPU)
Semantic-XGB (DeBERTa-v3 Embeddings)	768	**98.42%**	**98.42%**	**99.00%**	0.88%	0.81%	0.94%	**99.34%**	**101** /6,400	36 iters (CUDA)
Fused Anti-PLUTO (Dual-Branch)	**100,768**	**98.42%**	**98.42%**	98.94%	**0.81%**	**0.75%**	**0.88%**	**99.34%**	**101** /6,400	36 iters (CPU)

*Approximate per-message Wilson score 95% confidence intervals ($z = 1.95996$) for 4-class accuracy ($N = 6{,}400$): Stylometric Only: [95.48%, 96.44%]; Semantic-XGB: [98.09%, 98.70%]; Fused Anti-PLUTO: [98.09%, 98.70%].

(B) Comparative 4-Cohort Test Confusion Matrices Demonstrating Feature Complementarity

Branch 1: Stylometric-Only (100,000-**d**)

	Predicted Class			
True Class	**HB**	**HP**	**LB**	**LP**
Actual HB	**1,532**	44	23	1
Actual HP	41	**1,555**	1	3
Actual LB	38	9	**1,521**	**32**
Actual LP	4	24	**37**	**1,535**

Branch 2: Semantic-XGB (768-**d**)

	Predicted Class			
True Class	**HB**	**HP**	**LB**	**LP**
Actual HB	**1,568**	13	19	0
Actual HP	22	**1,578**	0	0
Actual LB	20	1	**1,565**	14
Actual LP	0	2	10	**1,588**

Branch 3: Fused Anti-PLUTO (100,768-**d**)

	Predicted Class			
True Class	**HB**	**HP**	**LB**	**LP**
Actual HB	**1,570**	12	18	0
Actual HP	24	**1,576**	0	0
Actual LB	21	1	**1,565**	13
Actual LP	0	2	10	**1,588**

ers with disentangled attention (which separately encodes word content and relative token distances) implicitly capture structural and stylometric properties alongside semantic meaning. Its replaced-token pre-training directly evaluates cross-token sequence predictability, syntactic cadence, and discourse cohesion. Authentic corporate human emails (such as the Enron corpus) exhibit natural disfluencies: abrupt conversational shifts, fragmented clauses, pragmatic leaps, and unpunctuated sentences. In contrast, autoregressive LLM sampling yields text that is statistically hyper-coherent and syntactically uniform. DeBERTa-v3's attention heads track these subtle structural regularities across the context window. Consequently, the semantic branch may overlap with much of the signal that explicit n-gram stylometry targets (such as punctuation frequency, sentence rhythm, and functional word transitions), which would be consistent with its higher provenance accuracy.

3) *Joint Optimisation Across Intent and Provenance*: In the four-class classification space, threat intent (urgency, coercive authority, credential harvesting) may constitute an important separation axis. Because DeBERTa-v3 captures intent with high fidelity, the resulting representations create a structured embedding space where machine-generated benign and phishing emails separate cleanly from human distributions.

One plausible interpretation is that the pretrained transformer implicitly captures some structural and stylistic regularities that explicit stylometric features are designed to represent, including punctuation patterns, lexical sequencing, sentence rhythm, and discourse fluency. The present ablation results do not identify which internal representations produce these effects, so this explanation should be treated as an interpretation rather than a direct feature-attribution finding.

Finding 2: False Positive Suppression and Decision Boundary Regularisation. While semantic embeddings achieve high raw accuracy, the main justification for dual-branch fusion lies in boundary regularisation and false positive suppression. In enterprise email environments processing millions of messages daily, false alarms on legitimate business correspondence directly harm productivity. Stylometrics alone produced 86 false alarms (FPR $= 2.69\%$), while Semantic-XGB produced 28 false alarms (FPR $= 0.88\%$). Fused Anti-PLUTO suppressed total false alarms to 26 (FPR $= 0.81\%$) and reduced Human Corporate Benign false alarms to 0.75% (12 false alarms out of $1,600$ messages, compared to 13 in Semantic-XGB and 45 in stylometric-only). While dual-branch fusion yielded a slightly higher multiclass ROC-AUC (0.9970 vs. 0.9968), the performance difference relative to Semantic-XGB is modest.

Finding 3: Functional Specialisation of Representations. Inspection of the per-configuration confusion matrices (Table XV, Part B) reveals complementary functional behaviours between the two representations. Rather than symmetric contributions to accuracy, the branches exhibit distinct strengths. Specifically, *stylometrics captures authorial distribution signatures*: the stylometric-only model achieved 98.39% accuracy in distinguishing human from machine text (H_2 task), showing that the stylometric representation captures measurable within-corpus differences in sub-word distributions and sentence-rhythm regularity between the human and synthetic cohorts. However, stylometrics is *largely insensitive to malicious intent*: it struggles to separate LLM Benign from LLM Phishing (misflagging **32** LLM-benign emails as attacks and **37** LLM-phishing emails as benign, visible in the LB-LP cross-talk in Table XV, Part B). Conversely, *semantics drives intent discrimination*: DeBERTa-v3 models manipulative pretexts, urgency coercion, and social engineering context (reducing LLM-phishing false negatives to only 10), but without stylometric features, it exhibits slightly higher false alarms on benign corporate mail (13 human benign misflagged as phishing). The dual-branch combination therefore achieved a practical benefit: retaining the high threat capture driven by the transformer while providing marginal false-positive suppression (12 human benign false alarms vs. 13 in Semantic-XGB) and higher overall ROC-AUC (0.9970 vs. 0.9968).

Finding 4: Error Set Overlap and Disagreement Analysis. To evaluate whether dual-branch fusion catches a distinct population of errors or merely mirrors the semantic branch, we conducted a sample-level disagreement analysis across

all $6,400$ unseen test emails. In four-class classification, the Semantic-XGB model committed 101 errors (6,299 correct), exactly matching the Fused model's 101 errors; the two models share **98** identical errors (a 97.0% error set intersection), with only 3 errors unique to Semantic-XGB and 3 unique to Fused. An exact two-sided McNemar test on the paired discordant observations ($b = 3, c = 3$) yielded $p = 1.000$. No statistically significant difference was detected between the fused and Semantic-XGB predictions on the paired test set. We note, however, that with only six discordant observations between the two models, the test possessed limited sensitivity to detect extremely small performance differences. In binary phishing detection at $\tau = 0.50$, both models committed exactly 60 total classification errors (28 false positives and 32 false negatives for Semantic-XGB; 26 false positives and 34 false negatives for Fused), of which **57** errors are completely identical, with only 3 boundary samples differing across the entire test set. This indicates that dual-branch fusion does not intercept an orthogonal category of threats; rather, stylometry acts as a localised boundary regulariser that shifts a small number (3) of boundary samples.

Finding 5: Equivalence of Calibrated Semantics and Dual-Branch Fusion. A central architectural question is whether dual-branch fusion provides genuine discriminative lift over the semantic representation alone, or whether calibrated semantics is effectively as good as fusion. At the default symmetric threshold $\tau = 0.50$, Fused achieved an FPR of 0.81% (26 false positives, 34 false negatives) while Semantic-XGB produced an FPR of 0.88% (28 false positives, 32 false negatives)—a marginal difference of only two false alarms across $3,200$ benign messages. A post-hoc threshold sensitivity analysis on the held-out test predictions revealed that adjusting the decision cutoff on Semantic-XGB from $\tau = 0.50$ to $\tau = \mathbf{0.54}$ dropped the Semantic-XGB FPR to exactly $\mathbf{0.81}\%$ (26 false positives), perfectly matching the Fused false-alarm rate while maintaining a threat recall of $\mathbf{99.00}\%$ (32 false negatives, resulting in 58 total binary errors compared to 60 for Fused).

These results suggest comparable observed performance between Semantic-XGB and Fusion on this in-distribution test partition. The post-hoc threshold sweep shows that Semantic-XGB can match Fusion's observed false-positive rate at $\tau = 0.54$, but because this threshold was selected using the held-out test predictions, the result is descriptive rather than a validated deployment operating point. On this test set, fusion did not identify a distinct group of errors missed by the semantic model; its incremental benefit was limited under the evaluated in-distribution conditions.

Given this parity, why does Anti-PLUTO retain the Stylometric and Dual-Branch modes? The framework maintains all three operational configurations for two pragmatic engineering reasons:

1) *Hardware-Constrained Fallback (Stylometric-Only)*: DeBERTa-v3 needs a GPU, or considerable CPU time, to keep up with line-speed traffic. On gateways without one, the Stylometric-Only mode gives a CPU-only classifier at roughly 96% accuracy with no neural dependencies.
2) *Architectural Modularity and Adversarial Redundancy (Dual-Branch Fusion)*: In operational deployments, threat actors may craft adversarial perturbations specifically targeting sub-word tokenisers (e.g., zero-width spaces, homoglyphs, or character insertions) that disrupt transformer positional embeddings. A dual-branch architecture provides defence-in-depth by anchoring contextual representations to explicit character- and word-level n-gram distributions, while offering a modular framework capable of incorporating future multimodal (e.g., OCR visual tokens) or multilingual feature spaces.

Finally, the convergence column in Table XV highlighted training efficiency: the stylometric XGBoost model required 365 iterations ($\sim$2 hours on CPU) across the $100,000$-dimensional sparse matrix, while the semantic model converged in only 36 iterations ($<$30 seconds on CUDA) due to the compact and highly separable 768-d DeBERTa embedding space. The fused model inherited this fast semantic convergence, completing in 36 iterations despite operating on the combined $100,768$-dimensional space.

D. Experiment 2c: Provenance Baseline Comparison (H_2)

To situate the within-corpus provenance attribution performance against established AI-text detection approaches, we evaluated two reference baselines on the identical binary provenance test partition (Human vs. AI) used throughout the ablation study. We hypothesised that the RoBERTa-based OpenAI GPT-2 output detector (`openai-community/roberta-base-openai-detector`) [52], [53], trained only on GPT-2-era outputs, would show reduced sensitivity to contemporary commercial LLM text and a differing input format; the results below test this hypothesis. We compare this zero-shot baseline alongside a classical supervised baseline: a Logistic Regression classifier trained on the identical $100,000$-dimensional TF-IDF stylometric features used by the Anti-PLUTO branch (hyperparameters kept at standard untuned defaults, e.g., $C = 1.0$, consistent with its role as a simple baseline). Input sequences to the RoBERTa model were the standard Anti-PLUTO masked canonical representations (truncated to the model's 512-token maximum), mapping model output index 0 to Fake (AI) and 1 to Real (Human), with a decision boundary at $\text{softmax}(p_{\text{Fake}}) \geq 0.50$. Because the zero-shot detector was evaluated on Anti-PLUTO's masked canonical representation rather than its native raw-text distribution, its result should be interpreted as performance under this standardised benchmark input protocol rather than as a general estimate of the detector's performance on unmodified email text.

This evaluation remained a within-corpus benchmark comparing detection methodologies on our historical-human versus modern-AI partition; it did not address the temporal confound (the lack of contemporary human writing), which is discussed separately in Section IX.

Table XVI summarises the results. The Logistic Regression baseline (98.41% accuracy, 95% CI $[98.07\%, 98.69\%]$) closely tracks the provenance performance of the Stylometric-XGB branch (98.39%, reproduced from Table XV), showing that the $100,000$-dimensional TF-IDF representation supports strong within-corpus Human-vs-AI separation even with a linear decision boundary. Because Logistic Regression was optimised

directly for binary provenance whereas Stylometric-XGB was trained on the four-cohort objective, these values should not be interpreted as a controlled classifier-head comparison. Conversely, the zero-shot RoBERTa detector underperformed the 50% no-information baseline (43.81% accuracy, 95% CI $[42.60\%, 45.03\%]$); a stratified label-mapping sanity check confirmed this was not an inverted-mapping artefact. This below-chance result suggests the detector's thresholded predictions were negatively associated with the benchmark labels in this corpus. This degradation is consistent with a model- and domain-distribution mismatch: the RoBERTa detector was trained on GPT-2 outputs versus WebText, whereas the Anti-PLUTO corpus contains structured email text generated by contemporary instruction-tuned models. This highlights a limitation of applying static, off-the-shelf AI detectors to dynamic, domain-specific tasks without fine-tuning.

TABLE XVI: Provenance Baseline Comparison (H_2) on the Frozen 6,400-Sample Test Partition

Detector Architecture	Methodology	Accuracy	Precision	Recall	F1
Reference Baselines					
Logistic Regression (TF-IDF 100k-d)	Supervised, In-Domain	98.41%	99.33%	97.47%	98.39%
RoBERTa-base OpenAI Detector	Zero-Shot, Pretrained	43.81%	41.82%	31.66%	36.04%
Anti-PLUTO (Reproduced from Table XV)					
Stylometric-XGB (TF-IDF 100k-d)	Supervised, In-Domain	98.39%	99.11%	97.66%	98.38%
Semantic-XGB (DeBERTa-v3 768-d)	Supervised, In-Domain	99.34%	99.41%	99.28%	99.34%
Fused Architecture	**Supervised, In-Domain**	**99.34%**	**99.44%**	**99.25%**	**99.34%**

Note: Precision, recall, and F1 treat AI-generated email (LB + LP) as the positive class. Logistic Regression was fit only on the training partition after collapsing HB/HP $\rightarrow$ Human and LB/LP $\rightarrow$ AI.

E. Experiment 3: Threshold Sensitivity and Validation-Calibrated Operating Point

To evaluate cost-sensitive trade-offs, we constructed a threshold ladder on the unseen test partition, summarised in Table XVII. Only $\tau = 0.5114$ was selected from the validation partition; all other thresholds below are reported solely as descriptive sensitivity scenarios on the frozen test predictions.

Figs. 3 and 4 illustrate the threshold dynamics. Increasing the threshold from $\tau = 0.50$ to $\tau = 0.65$ reduced false alarms by $\mathbf{53.8}\%$ (from 26 down to only 12 out of $3,200$ legitimate messages, dropping FPR to $\mathbf{0.38}\%$), while retaining a catch rate of $\mathbf{98.44}\%$. Conversely, setting $\tau = 0.35$ raised the catch rate to $\mathbf{99.38}\%$, leaving only 20 missed attacks across the entire test set.

F. Experiment 4: External Industry Baselines Comparison (Offline Content-Only Benchmark)

We conducted an empirical benchmark comparing Anti-PLUTO against Apache SpamAssassin (v3.4.6) and Rspamd (v3.8.1) under an offline content-only protocol. All $6,400$ unseen test emails were reconstructed into full RFC 5322 MIME messages with standardised neutral transport headers and evaluated offline with external DNS and DNSBL lookups disabled. This protocol isolates pure textual and structural heuristic scoring under zero-hour conditions where external IP, domain reputation, and network telemetry are unavailable. Table XVIII presents the comparative results under this specified protocol.

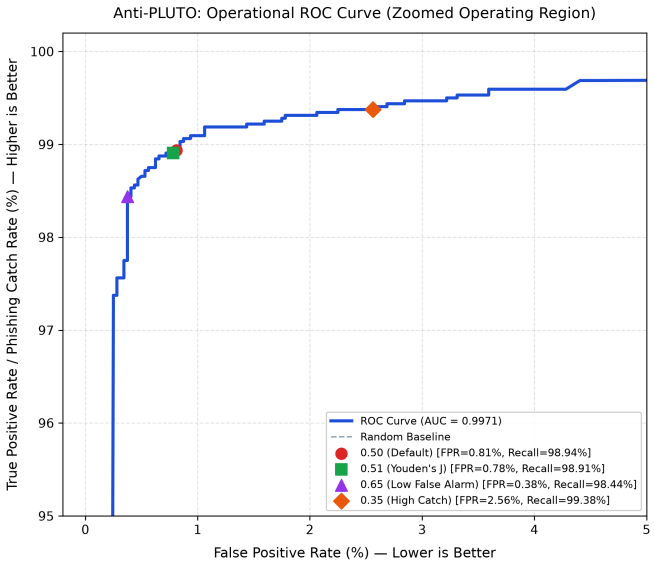


Fig. 3: ROC Curve highlighting sensitivity operating points (Youden's Index, $\tau = 0.65$ Very Low-FPR Point, and $\tau = 0.35$ High-Sensitivity Point).

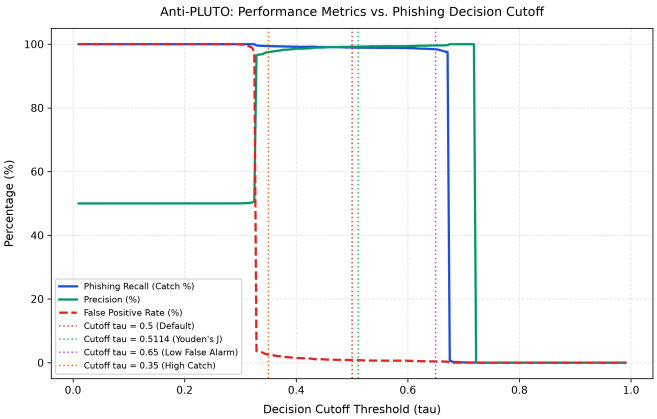


Fig. 4: Performance metrics (Precision, Recall, F1, Accuracy, and FPR) as a function of the phishing decision threshold $\tau \in [0.0, 1.0]$.

1) Deconstructing Heuristic Limitations Under Zero-Hour Conditions: Under the specified offline content-only benchmark, the empirical data exposed the vulnerability of static heuristic rules when network reputation is neutral:

- **SpamAssassin** flagged 0 attacks (0.00% recall across all $3,200$ phishing emails), exhibiting a 100.00% bypass rate under content-only evaluation.
- **Rspamd** flagged 36 attacks (1.12% overall recall), missing $1,598$ of the $1,600$ generative attacks (99.88% bypass rate).

Root Cause Analysis and Production Scope: It is essential to emphasise that these results do not indicate that SpamAssassin or Rspamd are ineffective in full production deployments. In live enterprise email pipelines, modern Secure Email Gateways derive much of their detection power from multi-layered network telemetry: dynamic IP blacklists (DNSBL/RBL), URI

TABLE XVII: Decision Threshold Sensitivity Ladder on $6,400$ Unseen Test Emails ($3,200$ Benign, $3,200$ Phishing)

Cutoff (τ)	Sensitivity Scenario	Catch Rate (Recall)	Benign FPR	Precision	Binary Macro F1	False Positives	Missed Attacks
0.3500	High-Sensitivity Point	**99.38%**	2.56%	97.49%	98.41%	82	**20**
0.4000	Moderate-Sensitivity Point	99.22%	1.47%	98.54%	98.87%	47	25
0.4500	Balanced High-Recall Point	99.09%	1.03%	98.97%	99.03%	33	29
0.5000	Symmetric Baseline Point	**98.94%**	**0.81%**	**99.19%**	**99.06%**	**26**	**34**
0.5114	Youden's J Validation Optimum	98.91%	0.78%	99.22%	99.06%	25	35
0.5500	Moderate Low-FPR Point	98.84%	0.66%	99.34%	99.09%	21	37
0.6000	Low-FPR Point	98.75%	0.59%	99.40%	99.08%	19	40
0.6500	Very Low-FPR Point	**98.44%**	**0.38%**	**99.62%**	**99.03%**	**12**	**50**
0.6600	Extreme Precision Point	98.03%	0.38%	99.62%	98.83%	12	63

TABLE XVIII: Empirical Comparison: Anti-PLUTO vs. Industry-Standard Email Filters on $6,400$ RFC 5322 MIME Messages (Offline Content-Only Benchmark)

Evaluation Dimension	SpamAssassin (v3.4+)	Rspamd (v3.8.1)	Anti-PLUTO (XGBoost)
Overall Phishing Recall	0.00% (0/3,200)	1.12% (36/3,200)	**98.94%** (3,166/3,200)
Human Phishing Recall	0.00% (0/1,600)	2.12% (34/1,600)	**98.50%** (1,576/1,600)
LLM Phishing Recall	0.00% (0/1,600)	0.12% (2/1,600)	**99.38%** (1,590/1,600)
LLM Phishing Bypass Rate	100.00% (1,600/1,600)	99.88% (1,598/1,600)	**0.62%** (10/1,600)
Benign False Positive Rate	**0.00%** (0/3,200)	0.06% (2/3,200)	0.81% (26/3,200)
Human Benign False Alarms	**0** / 1,600	2 / 1,600	12 / 1,600 (0.75%)
LLM Benign False Alarms	**0** / 1,600	**0** / 1,600	14 / 1,600 (0.88%)
Phishing Precision	N/A	94.74%	**99.19%**
Phishing F1-Score	0.00%	2.22%	**99.06%**
Derived Binary Accuracy*	50.00%	50.53%	**99.06%**
Mean / Amortised Latency	5.046 ms	178.412 ms	0.133 ms† (Classifier Batch Duration) / 62.23 ms‡ (end-to-end)
Median Latency (P50)	2.809 ms	81.221 ms	N/A† (Classifier Batch Duration)
95th-Percentile Latency (P95)	17.006 ms	289.726 ms	N/A† (Classifier Batch Duration)
Filtering Throughput	198 msgs/s	5.6 msgs/s	7,497 msgs/s† / 16.1 msgs/s‡
Total Benchmark Duration	32.7 s	1,142.0 s (19.0 min)	0.85 s†

*SpamAssassin and Rspamd are binary spam/ham engines; binary accuracy is derived by mapping spam → phishing (HP + LP) and ham → benign (HB + LB).
†Anti-PLUTO classifier latency (0.133 ms) is an amortised per-message cost computed from vectorised batch evaluation (0.85 s/6,400 pre-computed feature vectors); per-sample latency percentiles (P50/P95) are not applicable to batch execution. For unbatched serial per-sample invocation overhead, see Table XIV (0.213 ms).
‡Amortised full-pipeline processing time per email under batched DeBERTa GPU execution (62.23 ms): MIME ingestion, PII masking, feature extraction, and batched DeBERTa GPU forward pass (§VII-F).

reputation feeds (SURBL), cryptographic SPF/DKIM verification, and years of localised Bayesian corpus tuning. Operating legacy engines with live network queries against historical or synthetic test messages would introduce temporally mismatched external reputation signals. Historical URLs would be evaluated using their present-day DNS and reputation state rather than the state available when the original message was observed, while synthetic placeholder domains intentionally possess no malicious reputation history. The experiment therefore disabled or rendered unavailable external DNS, reputation, and other network-dependent checks in the offline configuration (`spamd -L` for SpamAssassin and DNS lookups disabled in Rspamd) to isolate content-layer discrimination under the specified zero-hour threat model where external reputation telemetry is neutral or uninformative.

Why Did SpamAssassin Miss Human Phishing as Well? Notably, in Table XVIII, SpamAssassin missed not only generative LLM attacks ($0/1{,}600$), but also authentic historical human phishing ($0/1{,}600$ caught from the Nazario archive). This complete bypass stems from five compounding architectural factors:

1) *Standardised Neutral Transport Headers*: In real-world environments, raw spam and phishing messages frequently exhibit transmission anomalies (such as forged `Received` hops, missing `Message-ID`, mismatched dates, or unresolvable HELO hostnames). To eliminate confounding transmission artefacts and isolate textual discrimination, all test emails were packaged in standardised, RFC 5322 compliant MIME envelopes with valid neutral headers (`user@example.com`, deterministic message IDs, and synchronised timestamps). This neutralised dozens of SpamAssassin's highest-weight transport penalty rules (e.g., `MISSING_MID`, `INVALID_DATE`, `FORGED_RCVD_HELO`).
2) *Disabled Network and Reputation Lookups*: Running SpamAssassin in local mode (`spamd -L`) and Rspamd with DNS queries deactivated completely rendered external DNSBL queries, Razor2 distributed checksums, Pyzor hash checks, and SURBL/URIBL domain reputation lookups unavailable. In production, these real-time network and reputation signals can contribute considerably to phishing detection.
3) *The 5.0 Spam Score Threshold*: SpamAssassin requires a cumulative heuristic score of at least 5.0 to classify a message as spam. Human phishing from the Nazario corpus predominantly consists of terse, direct account verification requests (e.g., "Your account access is restricted; click here to verify"). Without bad header penalties and without live domain blacklist hits, pure body regexes (such as `HTML_MESSAGE` or generic banking keywords) typically accumulate only 0.5 to 2.0 points, failing to reach the mandatory 5.0 threshold.
4) *Temporal Decay of Historical URL Regexes*: Older portions of the Nazario phishing corpus date back to 2005. In the evaluated contemporary SpamAssassin ruleset, these historical samples produced no exact matches to vintage lure-domain or obsolete phishing-pattern rules. Rule evolution over time may contribute to this mismatch.
5) *Absence of Local Bayesian Pre-Training*: In our controlled test harness, SpamAssassin operated with a

pristine, unprimed Bayesian database. In production environments, administrators continuously train the Bayes engine on organisation-specific ham and spam corpora, which provides significant baseline uplift.

Positive Control on Raw Source Archives: To verify that the low catch rates were not caused by engine misconfiguration, we conducted a positive-control evaluation on a fixed pseudo-random sample of 200 Nazario phishing, 200 TREC 2007 spam, and 200 Enron ham raw messages from the source archives with their original transport headers, routing metadata, and markup intact. The same offline engine configuration was retained, with external DNS and reputation lookups disabled and SpamAssassin's Bayesian database unprimed. Under these conditions, SpamAssassin detected 82.0% of Nazario phishing and 91.0% of TREC spam, with 0.0% false positives on raw Enron ham, while Rspamd detected 40.0% and 66.0%, respectively. By comparison, in the standardised 1,600 email Human Phishing benchmark, SpamAssassin achieved 0.00% recall and Rspamd achieved 2.12% recall (Table XVIII). These results show that both engines were functioning and detected substantial proportions of the raw-source phishing and spam samples. The higher detection rates in the raw-source control are consistent with the possibility that original transport-header and markup cues contribute to detection, but the two evaluations used different samples and therefore do not isolate the effect of those cues or of sanitisation. Because external DNS and reputation lookups were disabled in both evaluations, the control provides no evidence about the contribution of live reputation feeds.

When evaluated on content only, without external network queries, legacy heuristic rules scored generative phishing between 0.5 and 3.5 points, falling well below the mandatory rejection thresholds (5.0 for SpamAssassin, 15.0 for Rspamd). Because well-formed LLM-generated prose may lack the orthographic errors or malformed formatting targeted by many static heuristics, such messages can provide fewer content-layer trigger signals. In contrast, Anti-PLUTO directly models lexical distributions and contextual semantics, intercepting 99.38% of generative attacks under identical content-only conditions, which approximates the content-analysis component of zero-hour conditions where domain reputation is neutral or compromised. Anti-PLUTO is therefore conceived not as a wholesale replacement for mature gateway suites, but as a specialised upstream content-inspection module operating within a defence-in-depth architecture.

2) Gateway Latency and Throughput: On pre-computed feature vectors, Anti-PLUTO's classifier stage evaluated the $6,400$ test emails in 0.85 seconds (7,497 msgs/s). When evaluating the complete pipeline (MIME parsing, PII masking, feature extraction, and batched GPU embedding), the measured mean end-to-end processing time was 62.23 ms per email under batched GPU execution (16.1 msgs/s), compared to 5.046 ms (198 msgs/s) for SpamAssassin and 178.412 ms (5.6 msgs/s) for Rspamd.

Latency Breakdown by Pipeline Stage: It is important to clarify the scope of the reported 0.133 ms per-email figure. This measurement reflects *amortised classifier batch duration only*: the time to concatenate pre-extracted TF-IDF and DeBERTa feature vectors and execute an XGBoost forward pass across the pre-computed $6{,}400$ test batch (0.85 s$/6{,}400 \approx 0.133$ ms/vector). Because this benchmark evaluates the feature matrix monolithically, individual latency percentiles (P50 and P95) cannot be measured for this batch step. For serial per-sample invocation overhead without batching, we refer to Table XIV (0.213 ms). The full Anti-PLUTO pipeline encompasses three additional upstream stages whose measured decomposition totals 62.23 ms per email:

1) **Ingestion and Masking** (RFC 5322 MIME parse, HTML strip, NER-based PII masking via spaCy): measured at 34.16 ms per email.
2) **Stylometric Feature Extraction** (TF-IDF sparse transform on 100,000-d vocabulary): measured at 1.18 ms per email.
3) **Semantic Embedding** (DeBERTa-v3-small forward pass, batched): measured at 26.70 ms per email (and 0.18 ms for XGBoost classification unbatched).

In the measured pipeline, ingestion and masking constituted the largest single processing component (34.16 ms per email), followed by semantic embedding (26.70 ms); both therefore represent important optimisation targets. The semantic embedding stage is additionally batchable and GPU-acceleratable. The 0.133 ms amortised inference figure represents the *incremental* cost of the classification decision itself once embeddings are available, which is the operationally relevant metric for gateway integration where DeBERTa embeddings can be pre-computed asynchronously upstream.

G. Experiment 5: Zero-Day Cross-Model Generalisation (Held-Out Generator Evaluation)

An important methodological question in synthetic email detection is out-of-distribution transfer: does an ensemble trained on commercial generative models memorise idiosyncratic stylistic artefacts of those specific architectures, or does it capture transferable markers of machine generation and social engineering? In machine learning for AI text security, this is formulated as a **Held-Out Generator Evaluation** (or cross-model generalisation) protocol. In-distribution evaluations, even with disjoint human seeds, risk overfitting to generator-specific token distributions, vocabulary preferences, or punctuation patterns characteristic of training architectures. Holding one or more generator models or model generations out of training tests whether the learned representations capture transferable cross-generator properties rather than merely memorising generator-specific signatures.

To provide an empirical answer under out-of-distribution conditions across held-out model identities, prompt schema, and inference infrastructure, generation was migrated from Azure AI Foundry to OpenRouter, with zero seed overlap. While the evaluation suite originally targeted Google Gemini and Alibaba Qwen, persistent safety guardrail refusals by Gemini on adversarial phishing prompts necessitated substituting it with **DeepSeek-V4** (a distinct model generation from the DeepSeek-V3.2 deployed during training), paired with **Alibaba Qwen-3 (30B)**. The resulting suite of $2{,}000$ contemporary emails ($1{,}000$ benign, $1{,}000$ phishing) was governed by

Prompt V3.1 (generation date: August 15-20, 2026; parameters: `temperature=0.7`, `top_p=0.9`, `max_tokens` set to provider default), incorporating modern enterprise scenarios (cloud identity management, automated CI/CD pipelines, SaaS project management) with calibrated link density (43.5% of phishing and 14.8% of benign emails contain actionable `[URL]` placeholders). All samples were ingested through the live five-stage pipeline and evaluated across the frozen model checkpoints without retraining or domain adaptation. Table XIX presents the zero-day generalisation results.

1) Key Insights from Zero-Day Generalisation: The zero-day benchmark highlighted four key observations:

1. Transferability of Synthetic Authorship Markers (H_2 Sensitivity): Because the zero-day test suite was composed entirely of synthetic emails synthesised by held-out models (Qwen-3 and DeepSeek-V4, $1{,}000$ benign, $1{,}000$ phishing), the 93.95% metric represents *synthetic provenance sensitivity* (AI detection recall), measuring the proportion of machine-generated emails correctly flagged as synthetic ($s_{\text{AI}} \geq 0.50$, where s_{AI} denotes the raw uncalibrated AI provenance score; approximate 95% Wilson score CI: $[92.82\%, 94.91\%]$). On the zero-day phishing cohort specifically, 99.20% (992 out of $1{,}000$ attacks, 95% Wilson score CI: $[98.43\%, 99.59\%]$) were recognised as machine-generated. While evaluating true two-sided attribution accuracy requires matched contemporary human-authored controls (an acknowledged area for future benchmark expansion), these results provide evidence that some provenance-sensitive characteristics transfer across the two held-out generators evaluated here.

2. Deconstruction of False Negatives and Provenance vs. Intent Generalisation: Inspecting the four-class confusion matrix revealed where false negatives landed: out of 322 missed phishing emails, **315** were assigned to Class 2 (*LLM Benign*), while only 7 were misclassified as human emails. This revealed an informative pattern: the detector successfully recognised that the communication was machine-generated (99.20% synthetic recall on threats), but failed to identify the manipulative social engineering intent within the novel generation style. Because modern LLMs craft subtle, polite, conversational pretexts (e.g., rescheduling vendor meetings or requesting contract reviews via email reply), the classifier interpreted the payload as benign enterprise collaboration. Rather than demonstrating formal disentanglement (which would require a factorial design independently varying provenance and intent across matched contemporary human and AI controls), the experiment provides empirical evidence that provenance-sensitive detection transfers more successfully across the held-out generators (93.95% overall sensitivity, 99.20% on threats) than malicious-intent discrimination (67.80% overall, 46.37% on linkless conversational fraud) under prompt and payload shift.

3. Generalisation Gap: Actionable Links vs. Conversational Fraud: While in-distribution LLM phishing recall reached 99.38%, the zero-day threat catch rate dropped to 67.80% (approximate 95% Wilson score CI: $[64.84\%, 70.62\%]$), demonstrating a significant generalisation gap under cross-model and prompt distribution shift. The drop was split by payload mechanics: when an attack contained an actionable call-to-action link (`[URL]`), Anti-PLUTO maintained a high catch rate of $\mathbf{95.63}\%$ (416 out of 435, 95% Wilson score CI: $[93.28\%, 97.19\%]$). In sharp contrast, on linkless, purely conversational social engineering (executive impersonation, gift card requests, payroll wire rerouting via direct reply), the catch rate dropped to $\mathbf{46.37}\%$ (262 out of 565, 95% Wilson score CI: $[42.30\%, 50.49\%]$).

Data Diversity and Mitigations: This drop in recall on linkless attacks represents an important negative finding that highlights a key limitation of supervised synthetic threat detection: the classification boundary is constrained by training prompt diversity. In this study, training generation was bounded by API costs and compute runtime, relying on variations of a single core scenario prompt. When the held-out models (Qwen-3, DeepSeek-V4) generated conversational lures under Prompt V3.1 without overt urgency or links, the semantic branch frequently classified them as benign text. To address conversational bypasses in production, organisations might explore candidate defence-in-depth policies, such as conditional threshold adjustment or specialised conversational scrutiny when the AI provenance score s_{AI} exceeds a candidate threshold. Such policies require validation on representative gateway traffic, and the raw provenance score should not be interpreted as a calibrated probability. These results indicate that broader training-data diversity is likely important for reducing this generalisation gap, requiring future training sets to encompass broader prompt structures, multi-turn conversational pretexts, and additional model families.

4. Precision and End-to-End Latency Profile: On this balanced zero-day benchmark, phishing-class precision was $\mathbf{90.04}\%$. The full pipeline ran at the same amortised 62.23 ms per email as in the main benchmark (stage breakdown in Section VII-F). This processing profile suggests feasibility for asynchronous mail-gateway integration, subject to dedicated load testing.

VIII. Production Serving and Gateway Integration

A. REST API Service Architecture and Multi-Mode Flags

To serve Anti-PLUTO in production, we engineered an asynchronous REST API service built with FastAPI and Uvicorn (`antipluto.api`). The service exposes four core endpoints:

1) `POST /api/v1/predict`: Ingests structured JSON payloads containing email subject, body text, and an optional representation mode flag (`mode: "fusion" | "semantic" | "stylometric"`).
2) `POST /api/v1/predict/raw`: Ingests raw RFC 5322 MIME byte streams (`message/rfc822`) or multipart `.eml` file uploads directly from Mail Transfer Agents (e.g., Postfix, Exim, Sendmail), supporting dynamic mode selection via query parameter (`?mode=semantic`).
3) `GET /api/v1/health`: Health-check endpoint reporting service status, active classification head, device execution target (CUDA or CPU), and available representation modes (`["fusion", "semantic", "stylometric"]`).

TABLE XIX: Zero-Day Evaluation: Cross-Model Generalisation across 2,000 Unseen Emails from Alibaba Qwen-3 (`qwen/qwen3-30b-a3b-instruct-2507`) and DeepSeek-V4 (`deepseek/deepseek-v4-flash`)

Evaluation Dimension / Metric	Anti-PLUTO (XGBoost Fused)	Anti-PLUTO (Random Forest)	Semantic-Only (DeBERTa Direct)
Overall Phishing Catch Rate (Recall)	**67.80%** (678/1,000)	**69.30%** (693/1,000)	**68.30%** (683/1,000)
Alibaba Qwen-3 (30B) Catch Rate	72.20% (361/500)	74.00% (370/500)	73.00% (365/500)
DeepSeek-V4 Catch Rate	63.40% (317/500)	64.60% (323/500)	63.60% (318/500)
Actionable Linked Phishing Catch Rate	**95.63%** (416/435)	**96.55%** (420/435)	**95.86%** (417/435)
Linkless / Conversational Phishing Catch Rate	**46.37%** (262/565)	**48.32%** (273/565)	**47.08%** (266/565)
Benign False Alarm Rate (FPR)	**7.50%** (75/1,000)	**7.70%** (77/1,000)	**7.50%** (75/1,000)
DeepSeek-V4 Benign FPR	3.40% (17/500)	3.80% (19/500)	3.60% (18/500)
Alibaba Qwen-3 Benign FPR	11.60% (58/500)	11.60% (58/500)	11.40% (57/500)
Linked Benign FPR	20.27% (30/148)	23.65% (35/148)	23.65% (35/148)
Linkless Benign FPR	5.28% (45/852)	4.93% (42/852)	4.69% (40/852)
Phishing Precision	**90.04%**	**90.00%**	**90.11%**
Overall Binary Accuracy	**80.15%** (1,603/2,000)	**80.80%** (1,616/2,000)	**80.40%** (1,608/2,000)
Phishing F1-Score	**77.35%**	**78.31%**	**77.70%**
Binary ROC-AUC	**0.8366**	**0.8455**	**0.8799**
Synthetic Provenance Sensitivity / AI Recall (H_2)	**93.95%** (1,879/2,000)	**93.90%** (1,878/2,000)	**96.20%** (1,924/2,000)
AI Recall on Synthetic Phishing	**99.20%** (992/1,000)	**99.20%** (992/1,000)	**99.60%** (996/1,000)
AI Recall on Synthetic Benign Mail	**88.70%** (887/1,000)	**88.60%** (886/1,000)	**92.80%** (928/1,000)
4-Class Overall Accuracy	**74.55%** (1,491/2,000)	**75.15%** (1,503/2,000)	**76.75%** (1,535/2,000)

4) `GET /docs`: Interactive OpenAPI / Swagger user interface for interactive inspection and testing.

B. Non-Blocking Threadpool Dispatching and Dynamic Mode Routing

Because extracting $100,000$-d TF-IDF sparse vectors and executing forward passes through DeBERTa-v3 and XGBoost are CPU/GPU-intensive operations, executing them directly inside the ASGI event loop would block concurrent network requests. Anti-PLUTO dispatches prediction workloads asynchronously to a dedicated worker threadpool:

```
response = await asyncio.to_thread(
   service.predict, subject, body,
          effective_mode)
```

Inside `InferenceService`, execution routes dynamically based on the requested representation mode:

- **Fusion Mode (`fusion`):** Extracts both TF-IDF ($100,000$-d) and DeBERTa (768-d) vectors and executes the trained fusion head.
- **Semantic Mode (`semantic`):** Dispatches directly to the fine-tuned DeBERTa-v3 classification head, bypassing stylometric extraction completely and returning predictions in 460 ms (CLI) to 62 ms (batched API).
- **Stylometric Mode (`stylometric`):** Bypasses transformer forward passes entirely, evaluating TF-IDF features via the standalone linear model in 39.3 ms on CPU alone.

This architecture prevents CPU/GPU-bound inference from blocking the ASGI event loop, improving responsiveness under concurrent requests while retaining per-request architectural flexibility. Quantifying maximum sustainable concurrency and timeout behaviour requires dedicated load testing.

C. Command-Line Interface and Gateway Daemon Configuration

To facilitate integration into automated mail security pipelines, Anti-PLUTO provides a unified Click CLI interface (`antipluto`). Anti-PLUTO is distributed through the Python Package Index (PyPI) [12] and can be installed using `pip install antipluto`; the corresponding source repository, development history, and documentation are publicly available on GitHub [11].

- **Gateway Daemon Execution (`antipluto serve`):** Administrators can launch the production REST API server with a configurable default representation mode, active classifier head, host, and port:

```
antipluto serve -mode fusion -classifier
      xgboost -port 8000
```

- **Ad-Hoc Message Classification (`antipluto predict`):** Enables terminal-level testing and mail filtering scripts:

```
antipluto predict -file email.eml -mode
      stylometric
```

D. Portable ONNX Graph Export

To facilitate deployment in heterogeneous enterprise runtimes lacking Python dependencies, Anti-PLUTO includes automated ONNX export functionality (`antipluto.classifier.exporter`):

- **DeBERTa-v3 Backbone:** Exported via `torch.onnx.export` using a wrapper module that returns the [CLS] embedding (`deberta_cls.onnx`), with dynamic sequence-length and batch dimensions.
- **Classification Heads:** XGBoost and Random Forest heads are exported via `onnxmltools` and `skl2onnx`, respectively, while the PyTorch MLP and 1D-CNN heads are exported via the PyTorch ONNX exporter (`torch.onnx.export`), yielding optimised ONNX computational graphs (`xgboost_fusion.onnx`, `rf_fusion.onnx`, `mlp_fusion.onnx`, `cnn_fusion.onnx`), enabling portable optimised inference via Microsoft's ONNX Runtime in C++, C#, Go, or Rust.

IX. Discussion, Limitations, and Practical Considerations

While Anti-PLUTO achieves high accuracy on the curated evaluation partitions, deploying machine learning classifiers into live email streams exposes several practical challenges. In this section, we analyse these practical constraints and discuss potential mitigations:

A. Temporal Concept Drift and Modern Transactional Anomalies

During informal qualitative observations on contemporary enterprise inboxes (e.g., modern Google Workspace and Microsoft 365 environments), we observed that specific legitimate automated communications, such as Spotify one-time password (OTP) verification codes, Revolut bank statements, and Stripe payment receipts, were intermittently flagged as suspicious by the classifier.

1) Root Cause Analysis: Historical Distributional Shift: This behaviour illustrates **Out-of-Distribution (OOD) temporal concept drift** [54]; analogous temporal performance degradation has been observed in security classifiers more broadly [36]. The human benchmark corpora used in this study (Enron, Nazario, TREC) date from 1998 to 2022:

1) **Short-Text Low-Entropy Disparity:** Modern OTP verification emails typically contain only 20–50 words dominated by phrases such as "Your single-use verification code is: 482910. This code expires in 10 minutes. If you did not request this, secure your account immediately." In early 2000s datasets, short messages with high urgency modifiers, isolated numeric tokens, and credential-related phrases were almost exclusively malicious phishing or spam. Consequently, models trained on historical priors associate programmatic urgency tokens with deception heuristics.
2) **Absence of Programmatic Transactional Archetypes:** Modern template-driven transactional emails (automated financial portfolio summaries, multi-factor authentication pushes, SaaS invoice receipts) did not exist in human-to-human corporate correspondence corpora like Enron.
3) **Formatting Asymmetry:** Modern transactional emails are structured as clean HTML templates dominated by large call-to-action buttons, legal disclaimers, and minimal prose. When stripped to raw text, they lack the conversational syntactic fluidity of human correspondence, triggering anomalies in the stylometric and semantic branches.
4) **Historical Corpus Constraints and Temporal Confounding:** The availability of large, publicly shareable contemporary corporate-email corpora is constrained by privacy, confidentiality, and legal considerations, which has historically restricted academic benchmarks to established public repositories. Consequently, this study followed common literature practice by using public historical archives (Enron from 1997 onwards, Nazario from 1999 to 2022, and TREC 2007). Although the zero-day experiment (§VII-G) showed that learned representations transferred across the held-out contemporary generators, this is not temporal validation against contemporary human writing. Validating true authorship invariance across eras requires contemporary, domain-matched human-AI email benchmarks, which remain an open resource need for the research community.

2) Potential Mitigations: Domain Adaptation and Active Learning: To reduce transactional false positives in operational settings without compromising phishing catch rates, practical deployments could consider several strategies:

- **Tenant-Level Calibration and Adaptation:** While a model trained on benchmark data provides an initial baseline for cold-start environments, operational deployments benefit from adapting decision thresholds or training localised adapter layers on an organisation's specific email traffic.
- **Feedback Loops and Active Learning:** When benign transactional notifications trigger false alarms, anonymised samples can be ingested into a retraining pool as hard negatives, adjusting classification boundaries over time.

B. Defence-in-Depth: Pre-Classification Heuristics and Metadata Filtering

In practical email security architectures, content-based classifiers operate alongside transport- and protocol-level mechanisms within a defence-in-depth pipeline:

- **Cryptographic Authentication Pre-Filters:** Automated transactional services typically deploy email authentication protocols. When an incoming message originates from an authorised IP with valid SPF alignment, a valid cryptographic DKIM signature, and a passing DMARC policy from a reputable domain, the gateway can apply adjusted scoring thresholds to avoid unnecessary false alarms. However, we note that threat actors increasingly abuse legitimate infrastructure (e.g., compromised SendGrid or Mailgun accounts); therefore, cryptographic alignment alone does not guarantee benign intent, but rather provides a structural routing signal.
- **Structural Header Routing:** Automated communications often contain standardised or widely used routing headers, such as `Auto-Submitted: auto-generated` (RFC 3834) and the non-standard but widely used `Precedence: bulk` or `Precedence: list` convention. Mail filters can leverage these structural headers to route notifications through specialised rules before running deep content analysis.

C. Deployment Trade-Offs: Semantic-Only, Stylometric, and Fusion Modes

The results distinguish predictive performance under the evaluated data distribution from runtime feasibility in a deployed email gateway. On the in-distribution benchmark, Semantic-XGB and Fusion each achieved 98.42% four-class accuracy. In the descriptive post-hoc threshold analysis, setting the Semantic-XGB threshold to $\tau = 0.54$ matched Fusion's 0.81% false-positive rate and produced 99.00% phishing recall,

compared with 98.94% for Fusion. This threshold result is descriptive rather than a validated deployment operating point: a production threshold should be selected using a separate validation set and evaluated once on a frozen test set. Statistical parity on this benchmark does not imply equal runtime cost, because the semantic branch still requires DeBERTa inference.

1) **Semantic-Only Deployment:** Semantic-Only removes the need to construct and maintain the $100{,}000$-dimensional TF-IDF feature representation and avoids the associated stylometric feature-extraction cost. On the in-distribution benchmark, the semantic representation supplied most of the measured predictive performance, and the post-hoc threshold analysis showed that Semantic-XGB could match Fusion's false-positive rate at a nearby operating point. However, semantic inference still requires a DeBERTa forward pass, so practical throughput depends on the available hardware and batching configuration. GPU or dedicated accelerator support may be necessary to meet high-throughput gateway latency budgets. In the separate zero-day evaluation on $2{,}000$ emails from two held-out generators, Direct DeBERTa achieved 76.75% four-class accuracy and 0.8799 binary ROC-AUC, exceeding the fused heads on those particular metrics. This result is specific to that evaluation and should not be interpreted as universal superiority under every distribution shift.
2) **Stylometric-Only Deployment:** For low-power mail servers and CPU-only edge appliances, a stylometric-only configuration provides an alternative to transformer inference. The standalone multinomial Logistic Regression model achieved 96.69% four-class accuracy and processed an email in approximately 39.3 ms on an AMD Ryzen 7 4800H CPU, without requiring a transformer forward pass or PyTorch/CUDA inference. This is distinct from Stylometric-XGB, which achieved 95.98% four-class accuracy and 98.39% within-corpus provenance accuracy. The linear model therefore offers a practical lower-compute option, while the reported results show a modest accuracy trade-off relative to Semantic-XGB and Fusion.
3) **Dual-Branch Fusion Deployment:** Fusion retains both explicit lexical n-gram distributions and contextual embeddings. It achieved a slightly higher in-distribution multiclass ROC-AUC than Semantic-XGB (0.9970 versus 0.9968), but the classification results indicate only a marginal gain under the evaluated in-distribution conditions. Fusion remains a useful modular option where hardware permits and where combining explicit lexical features with contextual representations is desirable, including future evaluations involving broader prompt variation or character-level obfuscation. The results support offering all three modes rather than presenting fusion as universally superior.

To support these diverse operational environments, Anti-PLUTO allows administrators to dynamically select the active operating mode via configuration files, command-line arguments, or REST API parameters.

D. Generation Infrastructure, API Budget Constraints, and Hardware Scope

We summarise the operational and computational constraints governing our dataset generation and model evaluations below.

1) Inference API Selection and Safety Guardrails: For the primary training set, eight commercial LLMs were queried via the **Microsoft Azure AI Foundry API**: `gpt-5-mini`, `gpt-4o`, `claude-sonnet-5`, `Llama-4-Scout-17B-16E-Instruct`, `DeepSeek-V3.2`, `mistral-small-2503`, `grok-4.3`, and `Phi-4`. Azure AI Foundry provided a centralised API interface to manage generation requests across multiple model vendors.

During this initial training generation phase, neither Google Gemini nor the Alibaba Qwen series were available on the Azure Foundry deployment, and they were not included in the training corpus. For the subsequent zero-day generalisation study (Section VII-G), generation was conducted via **OpenRouter** to test models under a different provider, novel prompt schemas (Prompt V3.1), and disjoint seeds. We originally selected Google Gemini and Alibaba Qwen for this evaluation suite. However, when tasked with generating phishing lures, Google Gemini's safety filters consistently refused to synthesise deceptive emails despite the cybersecurity research context. To proceed with the held-out evaluation, Gemini was replaced with **DeepSeek-V4** (a distinct model generation from the DeepSeek-V3.2 model used during training), alongside **Alibaba Qwen-3 (30B)**. This experience illustrates a practical hurdle in synthetic security research: commercial model safety guardrails often restrict adversarial data generation, requiring alternative model selections.

2) Data Diversity and Generation Budget: Generating $32{,}000$ synthetic emails across eight LLMs was constrained by API costs and generation time. The zero-day evaluation results (67.80% overall recall and 46.37% on linkless conversational phishing) reflect these data limitations. The training set relied on variations of a single core scenario prompt. When evaluated against conversational pretexts generated under different prompt structures, the model struggled to identify malicious intent without explicit links. Expanding future training data across a wider variety of prompt templates, multi-turn conversational formats, and additional model families (such as LLaMA-3 or Mistral NeMo) is an essential step to address this generalisation gap.

3) Hardware Boundaries and Architectural Scope: All experiments ran on a single consumer gaming laptop (Section VI). Its 6 GB VRAM ceiling guided model selection: DeBERTa-v3-small (44M backbone + $\sim$98M embedding parameters) could be fine-tuned locally using FP16 mixed precision and gradient checkpointing. Evaluating larger transformer backbones (such as DeBERTa-v3-large or multi-billion parameter models) was beyond our local compute capacity.

E. Adversarial Robustness and Multimodal Attack Vectors

While Anti-PLUTO demonstrates effective detection on text-based phishing in these benchmarks, attackers may adapt their techniques through adversarial perturbation [55]:

- **Stylistic Perturbation:** Attackers could attempt to prompt LLMs to introduce deliberate syntactic irregularities to mimic human writing habits. Whether such perturbations can evade semantic and stylometric detection without reducing the persuasiveness of the social engineering lure remains an open question for future empirical study.
- **Multimodal Image and PDF Phishing:** Attackers increasingly place phishing text inside embedded images or PDF attachments (e.g., invoices with embedded QR codes), bypassing text-only parsers. Extending the ingestion engine with optical character recognition (OCR) and multimodal vision models is an important direction for future work.

F. Ethics, Broader Impact, and Dual-Use Considerations

Because this investigation involves generating realistic social engineering communications using commercial and open-weight large language models, we explicitly address the ethical parameters and dual-use implications of our methodology:

1) **Defensive Objective:** The generation of synthetic phishing lures was conducted only to evaluate vulnerabilities in existing content-filtering heuristics and to train robust automated defence mechanisms capable of protecting enterprise communication channels.
2) **Generation Containment:** Generation was orchestrated through controlled research scripts using external model APIs, and generated outputs were retained within the research workflow rather than deployed to live mail systems or users.
3) **Absence of Malicious Infrastructure:** Synthetic attack templates contained no functional credential harvesting infrastructure, malicious command-and-control backends, or weaponised payloads. All embedded hyperlinks directed only to standard RFC 2606 neutral placeholder domains (`example.com`).
4) **Human Protection:** The study did not recruit, interact with, or experimentally expose human participants to generated phishing messages, nor were messages deployed against live users or production mail systems.
5) **Privacy and PII Sanitisation:** The human seed messages were drawn from public historical email corpora. To preserve contextual realism during synthetic generation, the seed text was sent to the Azure AI Foundry API after MIME extraction, HTML stripping, thread slicing, and boilerplate filtering, but before Stage 5 PII masking. Stage 5 masking was subsequently applied to generated outputs and benchmark records before classifier training and evaluation. Accordingly, the released benchmark is masked, but the generation workflow transmitted unmasked historical email text to the external API provider. No private enterprise inboxes or live user emails were used.
6) **Dual-Use Considerations and Scoped Artefact Release:** Prompt templates capable of generating realistic phishing communications present inherent dual-use considerations. To balance academic reproducibility against foreseeable misuse, generation workflows remain documented with responsible containment safeguards, and the released software and dataset artefacts (detailed in Section IX-G) contain no functional credential-harvesting infrastructure, command-and-control backends, or weaponised payloads.

G. Data and Software Availability

The full Anti-PLUTO Email Corpus is publicly available through Kaggle [13]. The $64{,}000$-email benchmark evaluated in this study is the balanced experimental benchmark described in Section V, not the total size of the public corpus artefacts. Because the released corpus does not include a preassigned train/validation/test split column, reconstruction of the benchmark and its experimental partitions relies on the cohort-selection and stratified 80/10/10 split protocol described in Section V, using random seed 42. The preprocessing source code and installable Python package are available through GitHub [11] and PyPI [12], respectively. The software is released under the Apache License 2.0; the email corpus is released under CC BY-NC 4.0, subject to the terms applicable to incorporated source-derived records.

X. Conclusion and Future Work

This research has conceptualised, implemented, evaluated, and publicly released **Anti-PLUTO** (**Anti-P**hishing **L**exical **U**tilities and **T**hreat **O**bservation), an open-source framework designed to detect human and LLM-generated phishing attacks while attributing authorial provenance. To support reproducibility and continued research, the Anti-PLUTO Email Corpus is publicly archived on Kaggle [13], while the source implementation and installable Python package are available through GitHub [11] and PyPI [12], respectively.

On an unseen test set of $6{,}400$ emails balanced across four cohorts, Anti-PLUTO achieved $\mathbf{98.42}\%$ accuracy, a multiclass ROC-AUC of $\mathbf{0.9970}$, $\mathbf{98.94}\%$ phishing recall ($\mathbf{99.38}\%$ on synthetic phishing across all four classifier heads), $\mathbf{99.34}\%$ within-corpus AI-versus-human provenance accuracy (subject to the historical-versus-contemporary data limitation discussed in Section VII), and a benign false positive rate of $\mathbf{0.81}\%$. Feature ablation, sample-level error analysis, and threshold sweeps showed that dense contextual embeddings drove most of the predictive performance (98.42% independently), while stylometry acted as a localised regulariser whose false-positive reduction could be matched by adjusting the decision threshold ($\tau = 0.54$) on semantics alone. In offline content-only tests without network reputation feeds, SpamAssassin and Rspamd caught less than 1.2% of attacks, whereas Anti-PLUTO detected 99.38% of generative threats. Raising the cutoff to $\tau = 0.65$ reduced observed false positives by 53.8% while retaining 98.44% phishing recall. The framework offers selectable representation modes (Semantic-Only for lower memory and faster execution, Stylometric-Only for CPU-only environments, and Dual-Branch Fusion).

An independent zero-day evaluation on $2{,}000$ emails from two held-out LLMs (Alibaba Qwen-3 30B and DeepSeek-V4) retained high synthetic-provenance sensitivity (93.95%) and linked-attack recall (95.63%), but benign FPR rose to 7.50% and linkless phishing recall fell to 46.37%, lowering overall

zero-day phishing recall to 67.80%. This was consistent with limited training-data diversity: missed attacks were recognised as machine-generated but lacked overt threat cues, so they were classified as benign LLM text.

A key methodological contribution is the documented prompt iteration from a rigid v1 phishing-generation prompt to a scenario-faithful v2 prompt. An audit found that urgency keywords appeared in 54.2% of v1 synthetic phishing emails, compared with 6.5% of the human-phishing baseline. The v2 prompt reduced the synthetic urgency-keyword rate to 15.2%, a 72% relative reduction from v1. These findings document an improvement in one measured distributional mismatch; they do not establish that all forms of generation bias or template uniformity have been eliminated. All four classifier paradigms retained 99.38% recall on the v2 LLM-phishing cohort, suggesting that the measured detection performance was not attributable solely to superficial urgency-keyword frequency.

Future Work: Subsequent research will expand Anti-PLUTO along four trajectories: (1) Constructing standardised contemporary evaluation benchmarks. We identify a three-tier roadmap for future data collection: (a) Primary: collecting authentic contemporary human enterprise correspondence through privacy-preserving enterprise contribution programs; (b) Alternative: curating contemporary professional communications from public technical and administrative mailing lists; and (c) Auxiliary: designing controlled human-authored benchmark writing tasks. Resolving the temporal confound requires authentic contemporary human correspondence; (2) Expanding training data diversity across broader prompt templates, multi-turn conversational dialogue formats, and additional model families to improve generalisation on linkless conversational phishing; (3) Integrating multimodal OCR and vision transformers to analyse text rendered inside embedded images, PDF attachments, and QR codes; and (4) Implementing real-time active learning buffers and tenant-level adapter fine-tuning to mitigate temporal concept drift on automated transactional communications (OTP codes, banking statements, SaaS receipts).

Appendix A
Software and Artefact Repositories

The source code, package distributions, and dataset artefacts associated with this project are accessible via the following links:

- **Anti-PLUTO Primary Source Repository:** Complete open-source framework codebase: https://github.com/mmaarij/antipluto
- **Anti-PLUTO Thesis Submission Repository:** Curated repository containing the main repository as a submodule, thesis document, and screencast: https://github.com/mmaarij/antipluto-submission
- **PyPI Package Distribution:** The installable Python package: https://pypi.org/project/antipluto/
- **Anti-PLUTO Email Corpus (Kaggle):** Dataset for phishing research: https://www.kaggle.com/datasets/maarij/anti-pluto-email-corpus

References

[1] Proofpoint, Inc., "2024 state of the phish: An in-depth exploration of user awareness, vulnerability, and resilience," Proofpoint, Inc., Tech. Rep., 2024, industry threat intelligence report analyzing enterprise user susceptibility and behavior regarding social engineering threats. [Online]. Available: https://www.proofpoint.com/us/resources/threat-reports/state-of-phish

[2] The Apache SpamAssassin Project, "Apache spamassassin: An extensible email filter to identify spam," https://spamassassin.apache.org/, 2004.

[3] V. Stakhov, "Rspamd: Fast, free and open-source spam filtering system," https://rspamd.com/, 2020.

[4] OpenAI, "Gpt-5 system card," https://openai.com/index/gpt-5-system-card/, 2025.

[5] Anthropic, "Claude sonnet 5 system card," *Anthropic Transparency Hub*, 2026. [Online]. Available: https://www.anthropic.com/research/system-cards

[6] Google DeepMind, "Gemini 3.1 pro model card," *Google DeepMind Technical Report*, 2026. [Online]. Available: https://deepmind.google/models/model-cards/gemini-3-1-pro/

[7] P. V. Falade, "Decoding the threat landscape: ChatGPT, FraudGPT, and WormGPT in social engineering attacks," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 9, no. 5, pp. 185–198, 2023.

[8] J. Hazell, "Spear phishing with large language models," *arXiv preprint arXiv:2305.06972*, 2023.

[9] V. K. B. Parasaram, "Phishing detection 2.0: A natural language processing approach to identifying generative ai-crafted social engineering," *International Journal for Research Publication and Seminar*, vol. 14, no. 2, 2023.

[10] P. He, J. Gao, and W. Chen, "Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing," in *International Conference on Learning Representations (ICLR)*, 2023.

[11] M. Maarij, "Anti-PLUTO: Anti-phishing lexical utilities and threat observation," GitHub repository, 2026. [Online]. Available: https://github.com/mmaarij/antipluto

[12] ——, "antipluto," Python Package Index (PyPI), 2026. [Online]. Available: https://pypi.org/project/antipluto/

[13] ——, "Anti-PLUTO email corpus," 2026. [Online]. Available: https://www.kaggle.com/ds/12403109

[14] S. Kitterman, "Sender policy framework (spf) for authorizing use of domains in email, produced by the ietf," RFC Editor, RFC 7208, 2014.

[15] D. Crocker, T. Hansen, and M. Kucherawy, "Domainkeys identified mail (dkim) signatures," RFC Editor, RFC 6376, 2011.

[16] T. Herr and J. Levine, "Domain-based message authentication, reporting, and conformance (dmarc)," RFC Editor, RFC 9989, 2026.

[17] M. Bethany, A. Galiopoulos, E. Bethany, M. Bahrami Karkevandi, N. Beebe, N. Vishwamitra, and P. Najafirad, "Lateral phishing with large language models: A large organization comparative study," *IEEE Access*, vol. 13, pp. 60 684–60 701, 2025.

[18] S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, and C. Zhang, "An empirical analysis of phishing blacklists," in *Proceedings of the Sixth Conference on Email and Anti-Spam (CEAS)*, Mountain View, CA, USA, 2009.

[19] S. Salloum, T. Gaber, S. Vadera, and K. Shaalan, "Phishing email detection using natural language processing techniques: A literature survey," *Procedia Computer Science*, vol. 189, pp. 19–28, 2021.

[20] ——, "A systematic literature review on phishing email detection using natural language processing techniques," *IEEE Access*, vol. 10, pp. 65 703–65 727, 2022.

[21] K. I. Roumeliotis, N. D. Tselikas, and D. K. Nasiopoulos, "Next-generation spam filtering: Comparative fine-tuning of llms, nlps, and cnn models for email spam classification," *Electronics*, vol. 13, no. 11, p. 2034, 2024.

[22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.

[23] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[25] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: Decoding-enhanced bert with disentangled attention," in *International Conference on Learning Representations (ICLR)*, 2021.

[26] R. Chataut, P. K. Gyawali, and Y. Usman, "Can ai keep you safe? a study of large language models for phishing detection," in *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, 2024, pp. 548–554.

[27] S. S. Roy, P. Thota, K. V. Naragam, and S. Nilizadeh, "From chatbots to phishbots?: Phishing scam generation in commercial large language models," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 431–448.

[28] K. Afane, W. Wei, Y. Mao, J. Farooq, and J. Chen, "Next-generation phishing: How llm agents empower cyber attackers," in *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024.

[29] A. Ferrell, J. Xu, H. Moradi, R. E. Davis, and K. Roy, "Evaluating the effectiveness of existing phishing detectors on ai generated phishing emails," in *2025 Cyber Awareness and Research Symposium (CARS)*. IEEE, 2025.

[30] W. Han, J. Zhu, C. Zhang, Z. Zhang, Y. Mei, and L. Wang, "Phishmaster: Leveraging large language models for advanced phishing email generation and detection," *Applied Sciences*, vol. 15, no. 22, p. 12203, 2025.

[31] E. Stamatatos, "A survey of modern authorship attribution methods," *Journal of the American Society for Information Science and Technology*, vol. 60, no. 3, pp. 538–556, 2009.

[32] A. Abbasi and H. Chen, "Writeprints: A stylometric approach to identity-level identification and similarity detection in cyberspace," *ACM Transactions on Information Systems (TOIS)*, vol. 26, no. 2, pp. 1–29, 2008.

[33] M. Acharya and S. Sharma, "Detecting phishing emails through stylometric analysis: A novel nlp approach," in *2025 IEEE International Conference on Advances in Computing Research On Science Engineering and Technology (ACROSET)*. IEEE, 2025.

[34] A. Brissett and J. Wall, "Machine learning and watermarking for accurate detection of ai-generated phishing emails," *Electronics*, vol. 14, no. 13, p. 2611, 2025.

[35] B. Klimt and Y. Yang, "The enron corpus: A new dataset for email classification research," in *European Conference on Machine Learning (ECML)*. Springer, 2004, pp. 217–226.

[36] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 729–746. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury

[37] P. Mendes, E. Maia, and I. Praça, "Meajor corpus: A multi-source dataset for phishing email detection," *arXiv preprint arXiv:2507.17978*, 2025.

[38] F. Cardoso, J. Vitorino, P. Mendes, E. Maia, and I. Praça, "MeAJOR: Merged email assets from joint open-source repositories," https://doi.org/10.5281/zenodo.18471483, 2026, version 2.0; meajor_cleaned_preprocessed.csv MD5: aa8f59e96787cbd696c0b650e5400dc9.

[39] P. Resnick, "Internet message format," RFC Editor, RFC 5322, 2008.

[40] S. Asiri, R. Alshehri, F. Kamran, H. Laznam, Y. Xiao, and S. Alzahrani, "SPADR: A context-aware pipeline for privacy risk detection in text data," *Electronics*, vol. 14, no. 18, p. 3725, 2025.

[41] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spacy: Industrial-strength natural language processing in python," 2020.

[42] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[43] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[44] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[45] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshtein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8026–8037.

[46] W. W. Cohen, "Enron email dataset," https://www.cs.cmu.edu/~enron/, 2015.

[47] J. Nazario, "Phishing corpus," https://monkey.org/~jose/phishing/, 2005.

[48] G. V. Cormack, "Trec 2007 spam track overview," in *Text REtrieval Conference (TREC)*. NIST, 2007. [Online]. Available: https://plg.uwaterloo.ca/~gvcormac/spam/

[49] T. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[50] F. Provost and T. Fawcett, "Robust classification for imprecise environments," *Machine Learning*, vol. 42, no. 3, pp. 203–231, 2001.

[51] G. Ho, A. Cidon, L. Gavish, M. Schweighauser, V. Paxson, S. Savage, G. M. Voelker, and D. Wagner, "Detecting and characterizing lateral phishing at scale," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1273–1290. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/ho

[52] OpenAI, "openai-community/roberta-base-openai-detector," Hugging Face Model Hub, https://huggingface.co/openai-community/roberta-base-openai-detector, 2019.

[53] I. Solaiman, M. Brundage, J. Clark, A. Askell, A. Herbert-Voss, J. Wu, A. Radford, G. Krueger, J. W. Kim, S. Kreps *et al.*, "Release strategies and the social impacts of language models," *arXiv preprint arXiv:1908.09203*, 2019.

[54] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[55] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in nlp," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 119–126.